

ESPECIFICACIÓN

El proceso de curse de una operación

Versión: 1.0 · **Fecha:** 22-09-2026 · **Sistema:** NEX Factoring · Pipeline Comercial

Este documento describe **el curse de una operación** tal como el negocio lo define: la máquina de estados por la que pasa un negocio de factoring desde que el inbound lo abre hasta que Tesorería gira, quién actúa en cada estado y qué produce cada evaluación. La unidad es la **operación** —el negocio de un cliente con sus deudores y su paquete de facturas—, no el documento tributario: `spec-ciclo-factura.md` ya sigue a la factura y explica la costura entre los motores, y este documento **no lo repite**, lo cita por sección. Tampoco es el spec de ningún motor: cada uno tiene el suyo y acá sólo se dice cuándo corre, qué recibe, qué devuelve y qué se versiona.

Lo que este documento hace, y ningún otro hacía, es poner frente a frente **el modelo del negocio** (normativo: las cláusulas M-01 a M-40 de la Parte IV) y **lo que el sistema hace hoy** (medido contra el fuente el 22-09-2026). Donde el modelo dice algo que el sistema no hace, se escribe con las palabras **pendiente** o **decisión abierta**; nunca se describe como si existiera.

Se lee en cuatro tiempos:

- **Parte I — el modelo.** Qué es el curse, la máquina de estados con el catálogo real de etapas, el mapa del camino y los actores.
- **Parte II — las etapas del proceso.** Una sección por etapa, cada una en tres bloques: qué dice el modelo, qué hace el sistema hoy, diferencias y pendientes.
- **Parte III — los invariantes.** Las reglas del vault que gobiernan el proceso, con su número y su gate.
- **Parte IV — la conciliación.** La tabla cláusula por cláusula, las contradicciones que hay que decidir y lo pendiente agrupado por motor.

La premisa que ordena el documento. El modelo del negocio es la **definición normativa**; el sistema es la **medición**. Cuando difieren, este documento no elige: registra la diferencia con su evidencia (la regla del vault por su número, la sección del spec o la condición del fuente por su nombre) y la lleva a la lista de decisiones. Todo monto es un peso entero; el millón es abreviatura de pantalla (`M$`), no una unidad de dato.

Parte I · El modelo

1. Qué es el curse y qué lo distingue de la evaluación de una factura

Cursar es comprar un paquete de facturas a un cliente: decidir que se compra, a qué precio, contra qué línea, con qué aprobaciones y por qué vía se gira, y dejar cada decisión con evidencia de actor y hora. La **operación** es la unidad de esa decisión: nace como oportunidad de un cliente (`spec-ciclo-factura.md` §1 «Qué hace»: una oportunidad por cliente, nunca por factura, porque el ejecutivo se asigna por cliente), se convierte en oferta cuando el ejecutivo elige documentos, se evalúa, se publica, el cliente la firma y la casa la integra al core.

La **evaluación de una factura** es otra cosa y tiene otro dueño: cada motor decide sobre su propia unidad —la factura en líneas y prorratio, el deudor en verificación y giro, la regla x el sujeto en otorgamiento, la operación en pricing (`spec-ciclo-factura.md` §0, tabla «Los siete motores»)— y la operación agrega esos veredictos. Tres consecuencias que el resto del documento usa:

- La operación **cambia de estado**; la factura **entra o sale** de ella. Un deudor que no confirma retira sus facturas y la operación encoge (§11); un rechazo firme del cliente pierde la operación entera (§13).
- Lo que un motor decide **se versiona como evidencia** de la operación (§8), y la evidencia no reserva ni decide la evaluación siguiente (`spec-ciclo-factura.md` §17).
- Las **compuertas** (excepciones justificadas, monto a girar positivo, OTG-02, VER-01, LIN-01, GIR-02) son de la operación: se comprueban sobre el paquete completo, factura por factura cuando corresponde (regla 41).

2. La máquina de estados de la operación

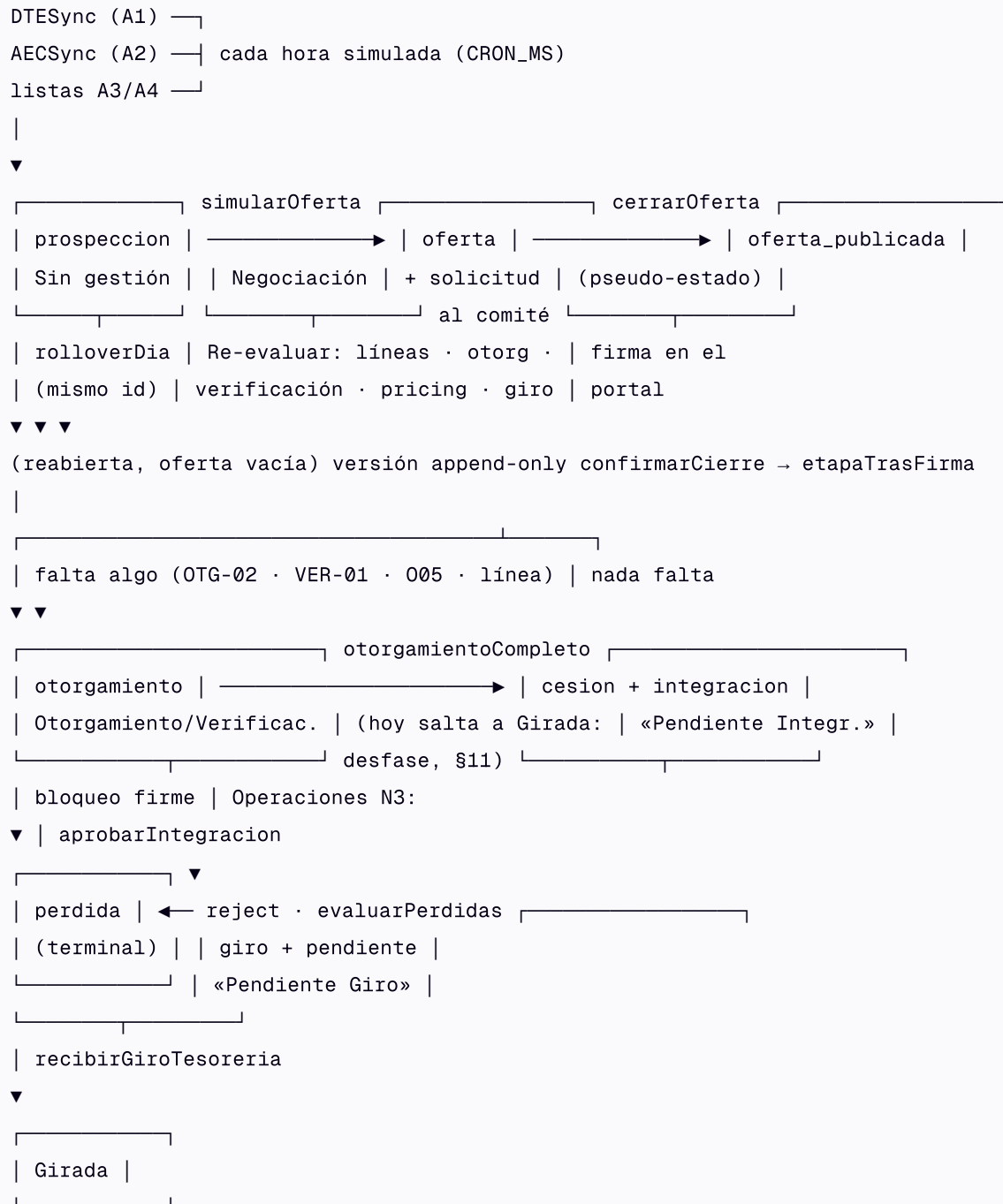
El catálogo real tiene tres capas. El **modelo** declara siete etapas en `STAGES` con su orden `STAGE_ORDER`. El **tenant** puede renombrarlas y colorearlas (`ETAPAS_TENANT_DEFECTO`, regla 28), y `ETAPA_PUBLICADA = "oferta_publicada"` es un pseudo-estado que no es un `stage`: es `oferta` con la oferta cerrada y publicada (`etapaVisualId`). Después de la firma manda un tercer traductor, `estadoOperacion`, que no es configurable y gana sobre el rótulo del tenant (`etapaDeDeal = estadoOperacion(d) || stageName(etapaVisualId(d))`).

ESTADO (ID DEL MODELO · RÓTULO DEL TENANT)	QUÉ LO ABRE	QUÉ LO CIERRA	QUIÉN ACTÚA
prospeccion · Sin gestión / Prospección	La corrida horaria del inbound crea la oportunidad con la oferta vacía (correrProceso); o el rollover de fin de día la reabre (rolloverDia)	La simulación de la oferta (simularOferta , regla 12-bis); o la pérdida	Inbound (sistema); el ejecutivo arma la oferta
oferta · Negociación / Oferta y Negociación	simularOferta promueve prospeccion → oferta	Cerrar y publicar (cerrarOferta); la pérdida por oferta no aceptada o cesión a otro factor (evaluarPerdidas)	Ejecutivo; Agente IA (opcional, regla 8); apoderados si hay pre-evaluación
oferta_publicada · Oferta publicada (pseudo-estado)	patchCierre asienta ofertaCerrada + ofertaComunicada (ofertaPublicada , regla 58). El predicado tiene dos lecturas más: «cerrada» = ofertaCerrada negocioNum y «comunicada» = ofertaComunicada o algún mensaje de waSession que calce /Oferta de factoring/i , porque el Agente IA publica por WhatsApp sin la bandera (regla 8, caso 158): también entra al pseudo-estado una oportunidad con negocioNum cuya sesión de WhatsApp contenga la oferta del agente, sin pasar por cerrarOferta	La firma del cliente en el portal (confirmarCierre); reabrir (reabrirOperacion) vuelve a oferta	El cliente en el portal; comité (solicitud inyectada); apoderados y verificación (pendientes ya exigidos, regla 54)
aceptadas · Aceptada	Id del catálogo; estadoOperacion lo rotula igual que cesion . En el fuente no se encontró escritor de stage: "aceptadas" con esa cadena: la firma escribe cesion u otorgamiento según etapaTrasFirma (el patch de confirmarCierre)	—	—
cesion · Aceptada (rótulo) / Pendiente Integración (con integracion: "pendiente")	etapaTrasFirma cuando la firma no deja nada pendiente: { stage: "cesion" , integracion: "pendiente" } ; también moverEtapa (botón «Avanzar» del detalle) tras la compuerta OTG-02, pero sin integracion — estadoOperacion la rotula «Aceptada», no «Pendiente Integración»— (§13)	aprobarIntegracion tras OTG-01 y controlesIntegracion	Operaciones N3

ESTADO (ID DEL MODELO · RÓTULO DEL TENANT)	QUÉ LO ABRE	QUÉ LO CIERRA	QUIÉN ACTÚA
<code>otorgamiento</code> · Otorgamiento / Verificación	<code>etapaTrasFirma</code> cuando falta algo: motivo <code>linea_o_deudor</code> · <code>excepciones</code> · <code>verificacion</code> · <code>evidencia</code> ; también <code>moverEtapa</code> y el arrastre del Kanban (<code>moveTo</code>), sin guarda (§13)	<code>otorgamientoCompleto</code> → el <code>useEffect</code> de avance; la pérdida por bloqueo firme	Apoderados por (área, nivel); Ejecutivo de verificación; el ejecutivo (respaldos); Operaciones (O05 física)
<code>giro</code> con <code>giroPendiente: true</code> · Pendiente de Giro	<code>aprobarIntegracion</code> escribe <code>{stage: "giro", integracion: "aprobada", giroPendiente: true}</code> y congela el giro en <code>repoGiro</code>	El callback de Tesorería (<code>recibirGiroTesoreria</code>)	Tesorería (fuera de NEX, regla 43)
<code>giro</code> sin <code>giroPendiente</code> · Girada	El aviso de Tesorería devuelve el patch <code>{giroPendiente: false, giroRef, giroTs}</code>	Terminal	—
<code>perdida</code> · Perdida	Ocho escritores (§13): manual (<code>reject</code>), cron (<code>evaluarPerdidas</code>), bloqueo firme (<code>useEffect</code>) y dos manuales sin causa (<code>moverEtapa</code> , <code>moveTo</code>)	Terminal (regla 5): reabrir es una operación nueva con referencia	Ejecutivo; sistema

Dos colapsos de pantalla que no son estados: `displayStageId` muestra `giro` como `aceptadas` en el tubo, y `fueraDelTubo` saca de la lista lo que ya es de Operaciones.

3. El mapa del camino



4. Los actores

ACTOR	QUÉ HACE EN EL CURSE	DÓNDE LO FIJA EL SISTEMA
Inbound (sistema)	Clasifica el stream, agrupa por cedente, abre o actualiza la oportunidad, cierra el día y reabre lo no gestionado	<code>tickCron</code> , <code>correrProceso</code> , <code>rolloverDia</code> ; <code>spec-inbound-facturas.md</code> §6
Ejecutivo comercial	Arma la oferta, pide la evaluación, justifica cada excepción, cierra y publica, puede reabrir, puede perder	<code>incorporarFacturasOferta</code> / <code>retirarFacturaOferta</code> (regla 33), <code>reevaluarLinea</code> , <code>solicitarAprobacionExc</code> , <code>cerrarOferta</code> , <code>reabrirOperacion</code> , <code>reject</code>
Agente IA	Canal opcional del primer contacto y de la publicación por WhatsApp; ofrece dentro del piso del deudor	<code>canalDeRegla</code> (<code>spec-inbound-facturas.md</code> §5.2), <code>SPREAD_MIN_DEUDOR</code> , regla 8
Apoderados por (área, nivel)	Aprueban o rechazan excepciones en la mesa Otorgamientos o en el tab del detalle; nivel igual o superior de la misma área	<code>aprobarExc</code> con OTG-01 antes de escribir; <code>puedeAprobarExc</code> (regla 18); <code>spec-gestion-excepciones.md</code> §2.2
Ejecutivo de verificación	Contacta al deudor, firma verificada / no verificada por factura, retira lo no confirmado	<code>verificarDeudor</code> , <code>DrawerVerificacion</code> (regla 53), <code>puedeVerificarFacturas</code> (regla 18)
Comité de líneas (sistema externo)	Resuelve la solicitud de líneas que el cierre inyecta; NEX sólo inyecta y consulta	<code>solicitudComiteDeOferta</code> → <code>api1Inyeccion</code> ; <code>api3EstadoProceso</code> ; regla 15
Operaciones N3	Visa 005 en la vía física; aprueba la integración al core tras los controles	<code>aprobarExc</code> (005), <code>aprobarIntegracion</code> con <code>controlesIntegracion</code> ; regla 41
Tesorería (fuera de NEX)	Gira y avisa	<code>recibirGiroTesoreria</code> (pura); regla 43
El cliente en el portal	Acepta formalmente firmando con código de un solo uso; es la única aceptación válida	<code>enviarCierre</code> , <code>confirmarCierre</code> ; regla 1

Parte II - Las etapas del proceso

5. Sincronización DTESync e inbound

Qué dice el modelo (M-01 a M-10). Las facturas llegan continuamente por DTESync junto con cesiones, notas de crédito, aceptaciones y reclamos. Cada hora los motores de inbound buscan facturas que cumplan sus condiciones; si detectan, abren una oportunidad al cliente o actualizan la abierta. La oportunidad cuantifica las facturas segmentadas en deudores Prime y Otros, y hace un join con la tabla de

líneas para decir cuántas facturas y cuánto monto tienen deudores con línea, Prime con línea y otros sin línea. Hay una hora de corte (23:00 por defecto) en que se eliminan las oportunidades no gestionadas y a las 06:00 del día siguiente se reinicia. Una factura es candidata si no está cedida a otro factoring, no está reclamada y no tiene nota de crédito que la anule; las reglas seleccionan por RUT emisor, fecha de emisión, reclamo, NC y cesiones previas, cruzando los emisores con una lista etiquetada.

Qué hace el sistema hoy.

- *Llegada.* `facturaDeDTE` lee del A1 las banderas `reclamada` (`EstadoDTE.Reclamado`), `notaCredito` y `folioNotaCredito`, y `credito` (`FormaPago === "2"`). Las cesiones **no** vienen por DTESync: llegan por el A2 AECSync y se cruzan por RUT cedente + folio (`spec-ciclo-factura.md` §1 «Qué recibe», §2 «Qué bloquea un documento»).

- *Corrida horaria.* Un `setInterval` cada `CRON_MS` (`cronMs: 3500`, «1 «hora» = N ms»; `frecuenciaMin: 60`) llama a `tickCron` → `correrProceso` + `evaluarPerdidas`; sigue tras

drenarse el stream (`spec-inbound-facturas.md` §8). Esa corrida es una **simulación en el navegador**: el `setInterval` sólo se instala si `CFG_ACTIVADA.modaDemo !== false` (su propio comentario: «El cron es parte del MOTOR DE SIMULACIÓN: sin modo demo no corre») y su período es `CRON_MS = cfgT.cronMs`, 3,5 s. `frecuenciaMin: 60` es un **parámetro huérfano**, igual que `horaInicio` / `horaFin`: se edita en Configuración > Operación (`<input {...num("frecuenciaMin", ...)}>`) y ningún cron lo lee (`frecuenciaMin` sólo aparece en su declaración, en la pantalla de Configuración y en un comentario). El job real del backend está pendiente (§10.4).

- *Candidatura.* `clasificarFactura` toma la **primera** regla activa de `INBOUND_RULES` cuyos criterios califican; un criterio desconocido califica todo (`facturaCalifica`). El

filtro de calidad «Buena factura» (cuatro condiciones, `CRITERIO_PRED`) y el catálogo de criterios viven en `spec-inbound-facturas.md` §3 y §5.1 y no se repiten acá. Lo que importa al proceso: **reclamo y NC no son criterios por sí solos** —una regla no puede seleccionar «reclamadas» ni «con NC», sólo excluirlas vía «Buena factura»—; los demás criterios de `CRITERIO_PRED` —incluido `Crédito`, que sí es seleccionable por sí solo, y los alias persistidos `Clientes` / `No clientes`, que existen por compatibilidad con las reglas guardadas en localStorage (el comentario de `CRITERIO_PRED`: un criterio desconocido califica TODO)— son los de la tabla de `spec-inbound-facturas.md` §5.1 y no se repiten acá. Las siete `INBOUND_RULES` usan sólo «Buena factura», las listas, Cliente / No Cliente y SOW.

- *Apertura y actualización.* `correrProceso` agrupa `acumulado` por cedente y crea la oportunidad

en `prospeccion` con ejecutivo, CAT, pool de facturas y **la oferta vacía** (`facturasOp: []`); tope `MAX_NUEVOS = 40` por corrida, el resto se re-encola. Si el cliente ya tiene una oportunidad `_inbound` en `prospeccion | oferta`, las facturas nuevas se suman a `facturasDisponibles` deduplicando por id/folio con la traza «Llegaron N factura(s) nueva(s)» (`aplicar(warn)`); la oferta elegida no se toca. Si la existente está aceptada, cursada o perdida, se abre otra (`spec-inbound-facturas.md` §6 punto 2). **Desfase medido**: el fuente promete lo contrario, pero no en la pantalla del detalle. El banner «Llegaron facturas nuevas del cliente. Se está recalculando la simulación con el paquete actualizado; los montos y condiciones pueden cambiar en unos segundos» es **inalcanzable**: está condicionado a `deal.actualizando && !fullPage` (lo mismo «Actualizando deudores de la oferta...») y el único sitio de render de `DealDrawer` pasa `fullPage` (el detalle es pestaña propia desde el 02-09-2026), así que es código muerto y la pantalla del detalle no anuncia nada. Lo que sí promete una re-simulación es el comentario de `correrProceso` («el resultado re-simulado llega tras una latencia»), el estado `actualizando: true` (que sólo dura la latencia) y los dos

mensajes de bitácora `logSys` «Recalculando N oportunidad(es) por M documento(s) nuevo(s) · latencia estimada ...» y «Recálculo aplicado»; pero `aplicar` sólo anexa a `facturasDisponibles` y apaga `actualizando` : sin `simulado` , sin `finanzasDe` , sin versión. Es la re-evaluación automática que M-12 pide y la regla 14 prohíbe; el comentario y la bitácora deben corregirse (y el banner muerto retirarse) o la decisión #1 de §15 contemplarlo (§16).

- *Segmentación Prime / Otros y join con líneas.* `capacidadDeudores` devuelve tres

tramos con `n` y `monto` : `primeConLinea` , `otrosConLinea` , `sinLinea` . Prime = `tipoDeudor` Lista Blanca o Deudor Autorizado (`CRITERIO_PRED` , `esPrime`). «Con línea» se decide en el nivel par (LF2/LF3 propia, o pozo LF4 / LF1 si prime) y en el nivel deudor (`lineaDeDeudor`) con el umbral «¿le cabe al menos una factura?» (`spec-ciclo-factura.md` §22). La tarjeta del tubo lo dibuja con `chipTramo` y el tooltip del chip `primeConLinea` lo declara **cota superior**, «no lo que se va a girar». El nivel cliente viene de `lineaCreditoDe` . Esa cuantificación **no es un dato de la oportunidad**: `correrProceso` no la escribe (no calcula prime/otros ni línea) y `capacidadDeudores` tiene un solo llamador, la tarjeta del tubo al dibujarse, donde es un lookup contra el listado A23 («no una corrida del motor»). En producción no habría dónde consultarla fuera de la pantalla.

- *Cierre del día.* No hay hora de reloj en la lógica, aunque sí como **parámetro huérfano**:

`CFG_OPER_BASE` declara `horaInicio: "08:00"` y `horaFin: "18:00"` con el comentario «ventana horaria de operación (inbound + actualizaciones)», Configuración > Operación las edita con `<input type="time">` y ningún otro sitio del fuente las lee (`grep horaInicio|horaFin` : cuatro resultados, todos de configuración). El cierre se dispara por conteo: `corridas % HORAS_DIA === 0` → `cerrarDiaRef.current` → `rolloverDia(nDia)` . Con `reaperturaDiaria: true` y `etapaNoGestionada: "prospeccion"` , toda oportunidad `_inbound` que quedó en esa etapa se **reabre con el mismo id** (comentario del fuente «EL ID NO CAMBIA»), con el paquete re-escalado, `facturasOp: []` , `simulado: false` y `reaperturas++` . Al llegar a `DIAS_SEMANA` el cron pausa y muestra el reporte semanal.

Diferencias y pendientes.

CLÁUSULA	ESTADO MEDIDO	QUÉ DIFIERE
M-01	implementado distinto	Las cesiones llegan por A2, no por DTESync; NC sin monto en el A1; «Aceptaciones» no existe como campo ni regla en ningún activo ni spec (el <code>EstadoDTE</code> del A1 trae sólo <code>Reclamado</code> , <code>NotaCredito</code> , <code>FolioNotaCredito</code> , <code>facturaDeDTE</code>); en el fuente sólo como dato sintético sin consumidor : <code>facturasDeCandidata</code> sortea <code>estado: "Aceptada" "Reclamada" "Sin acuse"</code> por RNG (única aparición de «Sin acuse»), que habría que retirar o conectar al A1 si el modelo confirma que la aceptación (acuse de recibo, Ley 19.983) es criterio de candidatura
M-05	implementado distinto	La cuantificación se calcula en el render de la tarjeta (<code>capacidadDeudores</code> , cuyo único llamador es la tarjeta del tubo), no se persiste en la oportunidad ni la produce el inbound. «Prime» acá es sólo listas; en verificación el protocolo recortado es prime o nota > 4,2 (regla 6). El modelo debe fijar cuál «Prime» segmenta la oportunidad
M-06	implementado distinto	Los tramos son <code>primeConLinea · otrosConLinea · sinLinea</code> (sin distinguir prime u otro en el último), y son cota superior, no asignación; se calculan en el render de la tarjeta, no en la oportunidad (M-05)
M-07	implementado distinto	Corte por N corridas, no por hora (la ventana <code>horaInicio</code> / <code>horaFin</code> es un parámetro declarado sin efecto); y <code>rolloverDia</code> reabre con el mismo id, no elimina. <code>spec-ciclo-factura.md</code> §17 dice que el rollover «cierra y re-origina las oportunidades del inbound que nadie gestionó» y que «Es una oportunidad nueva , con su propio identificador»: contradice al fuente («EL ID NO CAMBIA») y a la regla 22 (§15)
M-08	pendiente (con parámetro huérfano)	La ventana <code>horaInicio</code> / <code>horaFin</code> se edita en pantalla y nadie la consume: <code>rolloverDia</code> reabre en el mismo tick del cierre. Falta que el corte 23:00 y el reinicio 06:00 sean parámetros y que el job los use; hoy la ventana configurada es decorativa
M-09	implementado distinto	«Buena factura» no consulta <code>cedida</code> ; la cesión bloquea sólo al incorporar (<code>estadoCandidata</code>) y como pérdida por AECSync. Y la candidata exige «a crédito», que el modelo no menciona
M-10	implementado distinto	No existe criterio por fecha de emisión (el descarte < 8 días ocurre al cerrar , <code>intentarCerrar</code> ; al publicar «ya se resolvió al cerrar»), ni por cesión previa ; reclamo y NC sólo como parte de «Buena factura», no seleccionables; ni una lista de emisores con tags : los tags son del deudor (A3/A4) y del cedente sólo <code>esCliente</code> / SOW

6. La oportunidad y la selección del ejecutivo

Qué dice el modelo (M-11, M-12). El ejecutivo toma una oportunidad sin oferta, la abre y selecciona las facturas que cumplen los criterios de admisibilidad. Cada vez que selecciona o modifica su selección, la oferta se evalúa o re-evalúa.

Qué hace el sistema hoy. La oferta nace vacía por diseño (`spec-ciclo-factura.md` §1 «La oferta nace vacía»). El detalle es una pestaña propia con los documentos disponibles del libro de ventas del cliente (§2 «Documentos disponibles») partidos en «Deudores aprobados línea / sin aprobar» por lo que `seleccionConLinea` / `facturasConLinea` devuelven `CON_LINEA`. `estadoCandidata` bloquea en este

orden: `noConfirmada` (veto de verificación) → `notaCredito` → `reclamada` → `cedida` (con `nuestra` y `parcial`) → `otraOp`. Incorporar y retirar viven en `incorporarFacturasOferta` / `retirarFacturaOferta` (regla 33). Armar a mano tiene salida: «Tienes N factura(s) elegida(s) · M\$X» y la acción principal «Simular la oferta» (regla 13-sexdecies). Lo que saca la oportunidad de Prospección es la **simulación** (`simularOferta`, regla 12-bis), no editar el paquete.

La re-evaluación **se pide**: editar la oferta actualiza al instante sólo lo aritmético (monto, conteos, monto por deudor) y deja «Por evaluar», sin número, todo lo que depende de líneas o verificación hasta apretar Re-evaluar (`reevaluarLinea`; regla 14, gate 124, e2e-14).

Diferencias y pendientes. M-11 implementado. **M-12 es decisión abierta** y la primera contradicción de la Parte IV (§15): el modelo pide re-evaluación automática en cada cambio de selección; la regla 14 y tres specs (`spec-otorgamiento.md` §2, `spec-asignacion-lineas.md` §8.4, `spec-ciclo-factura.md` §3) fijan la explícita. No se cierra programando: es decidir si cada clic emite evidencia y consulta la API de líneas A23, y reemplazar la regla 14 por ADR.

7. La evaluación: cinco motores en paralelo

Qué dice el modelo (M-13). Al evaluar o re-evaluar la oferta corren en paralelo cinco motores — otorgamiento (EAN), verificación, líneas, giro y pricing—, cada uno con su spec.

Qué hace el sistema hoy. Son independientes: ninguno llama a otro y `girosDeDeal` es el único adaptador que conoce a los tres (`spec-ciclo-factura.md` §0 «Tres propiedades», punto 1). Pero **no hay una sola pasada**, y **no hay un evento de evaluación que invoque los motores**: `simularOferta` sólo escribe `simulado`, `stage` y `finanzasDe` sin llamar a `asignarLineas` ni a `verifDecision`; «Re-evaluar operación» (cabecera, `reevaluarLinea`) es un spinner de 700 ms que apaga `reevalPend` y con eso el render del detalle vuelve a correr líneas (`evalLin`), verificación (`verifFactura`) e ítems, sin emitir versión; «Re-evaluación de la simulación» (tab Otorgamiento, `reevaluarCliente`) pide variables frescas y emite versión; la Pre-evaluación pone la operación en la bandeja de los apoderados (`spec-gestion-excepciones.md` §4.1 y §5.5); el pricing se recalcula en el render del detalle (`tasaPond`). Las fichas dicen sólo lo que este documento necesita; los catálogos y las fórmulas viven en el spec de cada motor.

7.1 Otorgamiento (EAN)

Cuándo corre	Simulación (evalúa pero no emite versión : <code>spec-ciclo-factura.md</code> §23c fila 5; la v1 se emite recién con la primera «Re-evaluación de la simulación», <code>reevaluarCliente</code> ; <code>spec-gestion-excepciones.md</code> §4.1 la llama «versión v1» y está desactualizado), pre-evaluación, «Re-evaluar operación», «Re-evaluación de la simulación», cierre de la oferta (compuerta) y firma del cliente (<code>spec-otorgamiento.md</code> §2 lista cuatro momentos —pre-evaluación, una sola «Re-evaluación» disparada por «Re-evaluar operación», cierre y firma—: omite la simulación, no distingue los dos gestos que este documento separa y está desactualizado en un segundo punto: atribuye a «Re-evaluar operación» «veredicto vigente + versión nueva (evidencia)», cuando en el fuente ese botón (<code>lanzarReeval</code> → <code>onReevaluar</code> → <code>reevaluarLinea</code> ; cabecera del detalle) no toca <code>repoSimVersions</code> y la versión la emite sólo <code>reevaluarCliente</code> («Re-evaluación de la simulación»; el único emisor de <code>repoSimVersions</code> es <code>reevaluarCliente</code>); lo correcto es <code>spec-gestion-excepciones.md</code> §5.5 («No crea una versión») y §4.1)
Unidad de decisión	La regla × el sujeto: reglas C/O una vez por operación, reglas D una vez por deudor; <code>stKey</code> = <code>n</code> o <code>n@RUT</code> (<code>evaluarOtorgItems</code>)
Qué recibe	Operación (cliente, deudores, facturas, publicación, evidencia del contrato), variables del A16, catálogo, política, padrón (<code>spec-otorgamiento.md</code> §9.1)
Qué devuelve	<code>items[]</code> con disposición <code>aprobado · excepcion · rechazado · clasificacion · no_ejecutada</code> , área, nivel exigido tras el piso por monto, aprobadores; agregado <code>aprobada · sujeta · rechazada</code> (<code>visadoDealCalc</code>)
Qué se versiona	<code>vars</code> y <code>res</code> (disposición por regla) en <code>snapVersionCli</code> ; el visado no va en la versión, vive en <code>repoVisado</code> por <code>stKey</code> (§8)
Spec propio	<code>Specs_Procesos/Otorgamiento/spec-otorgamiento.md</code> ; el ciclo de la excepción en <code>Specs_Procesos/Excepciones/spec-gestion-excepciones.md</code>

Lo que el modelo llama bloqueante, re-evaluable y excepcionable existe con esos tres nombres: rechazo firme (`rechFirme`), rechazo re-evaluable (`rechReev`) y excepción con ruteo (área, nivel) y piso por monto (regla 4). Qué reglas son knockout y cuáles no se re-evalúan lo fija `spec-otorgamiento.md` §3.3 y `spec-gestion-excepciones.md` §2.4, no este documento. Una precisión de lectura del fuente: `NO_REEV_CLIENTE` es la lista de las **no** re-evaluables (`reglaReev = (n) => !NO_REEV_CLIENTE.has(n)`) y `rechReev` es su complemento; el literal inicial de `NO_REEV_CLIENTE` (n 12–41 del catálogo viejo) está muerto, porque la IIFE que inyecta el Modelo de Riesgo v1.0 hace `NO_REEV_CLIENTE.clear` y la rellena con los 28 ids del catálogo vivo: cuáles son lo dice `spec-gestion-excepciones.md` §2.4, no este documento. Los únicos tramos «rechazado» del catálogo vivo son `tHard` en C30/C31/C32: eso es lo que sostiene «rechazo firme sólo C30–C32».

7.2 Verificación de facturas

Cuándo corre	En el render del detalle (<code>verifFactura</code>), en la misma pasada que líneas (regla 14, gate 124), y con evidencia sólo en <code>snapVersionCli.verificacion</code> al hacer «Re-evaluación de la simulación»; ningún evento de evaluación la invoca: <code>simularOferta</code> no llama a <code>verifDecision</code> y «Re-evaluar operación» sólo apaga <code>reevalPend</code> (§7); al cerrar y publicar aparece el tab (regla 58)
Unidad de decisión	El deudor dentro de la operación; la verificación de una factura es la de su deudor (<code>verifDecision</code> , regla 6)
Qué recibe	El par cliente-deudor y todas las facturas de ese deudor en la operación (<code>verifFactura</code>); el estado del cliente por parámetro (regla 6, «Regla O» de verificación; <code>spec-verificacion-facturas.md</code> §4.0)
Qué devuelve	<code>requiere_verificacion</code> con motivo y la lista completa de criterios evaluados (<code>spec-verificacion-facturas.md</code> §2.3); por operación, <code>verifResumenDeal</code> cuyo <code>.pend</code> es VER-01
Qué se versiona	El veredicto por deudor en <code>snapVersionCli.verificacion</code> ; el contacto vive aparte en <code>repoVerifTel</code> (por factura: { <code>por</code> , <code>fecha</code> , <code>checklist{existencia, recepcion, fechaPago}</code> , <code>contacto</code> , <code>compromiso</code> , <code>respaldo</code> , <code>sinRespaldo</code> , <code>notas</code> } , <code>verificarDeudor</code>) y el veredicto congelado en <code>repoVerifVeredicto</code> (por deudor y <code>congelarVeredicto</code>)
Spec propio	<code>Specs_Procesos/Verificacion/spec-verificacion-facturas.md</code>

7.3 Asignación de líneas

Cuándo corre	En el render del detalle (<code>evalLin</code>) y en cada versión (<code>snapVersionCli</code> , sólo desde <code>reevaluarCliente</code> y <code>retirarFacturaOferta</code>), recálculo siempre completo (regla 7); no la invoca <code>simularOferta</code> (que sólo escribe <code>simulado</code> , <code>stage</code> y <code>finanzasDe</code>) ni «Re-evaluar operación» (<code>reevaluarLinea</code> , un spinner de 700 ms que apaga <code>reevalPend</code>); al cerrar la oferta para armar la solicitud (<code>cerrarOferta</code> , llamada a <code>asignarLineas</code>); LIN-01 factura por factura a integrar (regla 41)
Unidad de decisión	La factura completa : <code>CON_LINEA</code> o <code>REQUIERE_COMITE</code> , nunca parcial (<code>asignarLineas</code>)
Qué recibe	Facturas itemizadas, RUT cliente, estado de líneas inyectado (<code>inyecta</code>)
Qué devuelve	<code>cursable</code> , <code>requiereComite</code> , <code>facturas</code> con <code>origen[{lineaId, tipo, monto}]</code> (una factura sí puede repartirse LF3 → LF2), <code>deudores</code> , <code>lineasUsadas</code> , <code>solicitudes</code> con motivo <code>par · lf4 · lf1 · cliente · deudor · suspendida</code>
Qué se versiona	<code>snapVersionCli.linea</code> completa; <code>lineaDeVersion</code> pinta desde la última versión sin volver a correr el motor (regla 13)
Spec propio	<code>Specs_Procesos/Lineas/spec-asignacion-lineas.md</code>

La evaluación **no reserva**: la reserva la crea el sistema de líneas al firmar el cliente y el core la commitea al aprobar Operaciones (regla 12; `spec-asignacion-lineas.md` §3.7).

7.4 Giro

Cuándo corre	En cada re-evaluación (el chip ya aparece en la tarjeta del tubo desde que la oferta está simulada, <code>spec-ciclo-factura.md</code> §0); se congela en la inyección (<code>repoGiro</code> , regla 43)
Unidad de decisión	El deudor ; las facturas heredan (<code>asignarGiros</code> , regla 22)
Qué recibe	Facturas con su giro ya prorrateado, <code>verificado{deudor}</code> , <code>excepcionDeudor{deudor}</code> , <code>excepcionCliente</code> , <code>primeraOperacion</code> , <code>montoGirar</code> (<code>spec-modelo-giro.md</code> §6.1)
Qué devuelve	Montos y facturas por tipo GE / GN, fila por factura con hechos, <code>cuadra</code> / <code>descuadre</code> ; la suma por tipo es siempre el monto a girar
Qué se versiona	No va en <code>snapVersionCli</code> ; se congela aparte al inyectar (<code>giroCongelado</code> gana sobre el cálculo del día)
Spec propio	<code>Specs_Procesos/Evaluacion_Factura/spec-modelo-giro.md</code>

7.5 Pricing

Cuándo corre	Al simular (<code>simularOferta</code> → <code>finanzasDe</code>) y en el render del detalle (<code>proRiesgo</code> / <code>tasaPond</code>); el motor de conceptos <code>simularOperacion</code> es puro
Unidad de decisión	La operación para el desglose; el deudor para la tasa (<code>spreadSugerido</code> con piso <code>SPREAD_MIN_DEUDOR</code>); la factura para el prorrateo (<code>prorratearOperacion</code> , regla 21)
Qué recibe	Catálogo de conceptos y constantes del tenant por parámetro; política comercial (spread de lista, tabla SOW); costo de fondo; <code>tasaModo</code>
Qué devuelve	Filas del desglose y el cierre fijo del catálogo (<code>spec-pricing-simulacion.md</code> §3.1 «El cierre es fijo»; regla 20); tasa y plazo equivalentes (<code>tasaEquivalente</code>); giro por factura
Qué se versiona	Nada de las condiciones comerciales : ni <code>snapVersionCli</code> ni <code>huellaOperacion</code> contienen tasa, comisión ni anticipo (regla 23)
Spec propio	<code>Specs_Procesos/Evaluacion_Factura/spec-pricing-simulacion.md</code>

La comparación contra el último negocio existe y es del **cliente, no del par**: `historialComercial` siembra sólo por cliente (`hashStr(cliente + "hc")`) y `hashStr(cliente + "_op_" + i)`) y el parámetro `deudor` sólo rotula cada fila; `tasaUltimoNegocio` llama `historialComercial(deal.cliente, deal.deudor)` y toma `h[0]` («Hace 1 mes»), y el propio comentario del fuente dice «Tasa del ÚLTIMO negocio cursado del cliente». Es sintético, sembrado sólo por cliente. Y `CFG_ACTIVA.tasaModo` (`"riesgo" | "ultima" | "mayor"` , default `"mayor"` : «no ofertar bajo la última tasa cursada») decide cuál manda; si manda la del último negocio, el prorrateo pasa a top-down.

Diferencias y pendientes (M-13, M-14 a M-17, M-22 a M-27, M-30 a M-35). Las fichas de la Parte IV lo detallan; lo que importa a nivel de proceso: (a) tres gestos de evaluación y no uno; (b) el resultado de

líneas **no entra** al criterio de giro (LIN-01 bloquea la integración aparte, regla 41), contra M-33; (c) el pricing compara contra **el** último negocio del cliente (no del par) y no contra «los últimos negocios»: `spec-pricing-simulacion.md` §4.2 fija **el** último negocio, en singular, como tasa top-down, y lo que ningún spec define es una ventana de N negocios ni un promedio (M-35); (d) el modelo (M-22) dice «empresa emisora» para la unidad de verificación, mientras `spec-verificacion-facturas.md` §2.1 fija la unidad en el deudor («La decisión es por deudor dentro de la operación») y el sistema decide así (`verifDecision` ; `verifFactura` evalúa todas las facturas del deudor): decisión abierta (§15); (e) el contacto de verificación **sí registra** los tres hechos que el modelo pide obtener: `DrawerVerificacion` lleva el checklist `{existencia, recepcion, fechaPago}` (rótulos «Existencia de la factura», «Recepción conforme», «Fecha de pago»; en la mesa `CHECKS`) y exige los tres más la fecha de pago comprometida antes de confirmar (`completo = chk.existencia && chk.recepcion && chk.fechaPago && !!compromiso`), y `verificarDeudor` lo persiste por factura en `repoVerifTel` , de donde `verifFactura` lo lee (`checks`). Lo pendiente es sólo documental: `spec-verificacion-facturas.md` no describe el checklist (M-22-bis, §14 y §16).

8. El resultado versionado de una evaluación

Qué dice el modelo (M-36 a M-40). Al terminar los motores se obtienen, versionados: las condiciones comerciales de la oferta, los requisitos de excepción, los requisitos de verificación, la asignación de líneas y las solicitudes de líneas adicionales, y los montos a girar por giro normal o express. Las excepciones y verificaciones resueltas se almacenan versionadas y no se vuelven a pedir (M-20, M-25).

Qué hace el sistema hoy. Cada «Re-evaluación de la simulación» emite una versión append-only en `repoSimVersions` con `snapVersionCli` ; la simulación inicial **no** emite versión (`spec-ciclo-factura.md` §23c fila 5; `simularOferta` no toca `repoSimVersions`). La **primera** re-evaluación emite **dos**: `reevaluarCliente` hace `if (!vs.length) repoSimVersions.push(deal.id, snapVersionCli(deal, 0))` —«persiste la evaluación inicial (v1)», dice el comentario en `reevaluarCliente` — y luego la nueva. Esa v1 es **retroactiva**: se calcula con los datos del momento de la re-evaluación, no de la simulación, así que la evidencia de la evaluación inicial no es contemporánea (el único emisor es `reevaluarCliente`). El retiro por verificación emite una versión con la línea recortada (`retirarFacturaOferta`). Desde Aceptada en adelante una versión nueva no re-asigna: recorta la anterior (`recortarAsignacion` , llamada en `snapVersionCli`).

LO QUE PRODUCE LA EVALUACIÓN	DÓNDE QUEDA	SOBREVIVE A LA RE-EVALUACIÓN SIGUIENTE
Condiciones comerciales (tasa, comisión, anticipo, monto a girar)	Campos de <code>finanzasDe</code> en la operación; no en la versión ni en la huella (regla 23)	Se recalculan; ninguna evidencia las congela (M-36, implementado distinto)
Requisitos de excepción (ítems con disposición, área, nivel, aprobadores)	<code>snapVersionCli.res</code> (disposición por regla) + <code>repoVisado</code> / <code>VISADO_STATE</code> por <code>stKey</code> + <code>SOLICITUD_EXC</code>	La versión es foto; el visado sobrevive por clave estable (<code>reevaluarCliente</code>): «NO se tocan las excepciones ya resueltas». Si la regla deja de gatillar, el ítem sale aprobado y el visado, la solicitud, el hilo y la tarea quedan huérfanos en el repositorio: no se eliminan ni se marcan (M-21)
Requisitos de verificación (por deudor, con causas)	<code>snapVersionCli.verificacion</code> + <code>repoVerifTel</code> por (operación, factura) + <code>repoVerifVeredicto</code> por (operación, deudor)	El contacto y el veredicto congelado sobreviven por clave estable; <code>limpiarSimulacion</code> y reabrir no los tocan (regla 1). No es versionado: es clave estable (M-25)
Asignación de líneas y solicitudes	<code>snapVersionCli.linea</code> (por factura, con origen) + <code>repoSolicitudComite</code> al cerrar	La versión anterior es evidencia, no reserva ni entrada del cálculo (<code>spec-ciclo-factura.md</code> §17); el diff <code>gano_linea · perdio_linea · cambio_de_linea</code> sólo explica
Giro GE / GN	Cálculo del día (<code>giroDeal</code>) hasta la inyección; después <code>repoGiro</code> (<code>giroCongelado</code>)	Se recalcula en cada re-evaluación y se congela al inyectar (regla 43)
Huella del paquete	<code>huellaOperacion</code> : <code>op · rut · nd · nf · monto · deudores[rut:monto]</code> + SHA-256, en <code>repoContratoEvidencia</code>	Si el paquete cambia, la huella no calza y O05 vuelve a ser excepción por sí solo (regla 23)

Diferencias y pendientes. M-37, M-38, M-39, M-40 implementados. M-36 implementado distinto: lo que se versiona es el veredicto de los motores, no las condiciones comerciales, y la v1 no es contemporánea de la simulación. M-20 y M-25 implementado distinto: sobreviven, pero por clave estable (`stKey`, `facturaId`, deudor), no dentro de la versión; el spec debe decir cuál de los dos modelos de datos quiere (§15). M-21 implementado distinto: el «marcar como cumple» ocurre solo; el «eliminar la excepción» no ocurre (buscado «huérfan» —sólo en Configuración > Áreas, sobre criterios que rutean a un área borrada—, «ya no levanta», «ya no aplica», «visado obsoleto»: nada sobre visados obsoletos). `spec-modelo-giro.md` §7 todavía dice que no hay pantalla de la asignación mientras la regla 22 cita `ChipGiro` en tres sitios: uno de los dos está desactualizado.

9. Justificación de excepciones, cierre, envío a comité y publicación

Qué dice el modelo (M-18, M-19, M-28). Si la regla es bloqueante, la única forma de avanzar es eliminar las facturas del deudor, y si el deudor está bloqueado el sistema debe retirarlas automáticamente como resultado de la simulación. Tras el otorgamiento, el ejecutivo justifica cada regla excepcionable y sólo

entonces puede enviar a comité y publicar. Y, textualmente: «En caso de existir suficiente línea, al publicarse la oferta el sistema debe enviar a solicitar automáticamente al comité las líneas puntuales necesarias para cubrir los gaps». La lectura literal («existir») no cuadra con «cubrir los gaps», así que este documento lee M-28 como «en caso de **no** existir suficiente línea» —**errata del modelo, a confirmar por el usuario**—; el sistema sólo inyecta cuando `ev.requiereComite > 0`.

Qué hace el sistema hoy.

- *Justificar.* `solicitarAprobacionExc` **no exige nada**: recibe `(deal, x, execCode,

`comentario, archivos, sinComentarios)` y guarda `comentario || ""`, `(archivos || []).slice` y `!sinComentarios` sin validar. La exigencia «comentario, respaldo adjunto o la declaración explícita «sin comentarios»» vive en el formulario del tab (dos llamadas a `solicitarAprobacionExc`: la del formulario «con el comentario, los adjuntos y la declaración «sin comentarios»» y la de «solicitar todas», con comentario vacío y `sinComentario: true`); el botón de Pre-evaluación la llama para cada excepción pendiente con `""`, `[]` y `sinComentarios`: una solicitud sin justificación ni declaración. Al solicitar, escribe `SOLICITUD_EXC`, enciende la pre-evaluación, audita, abre el hilo a los aprobadores hábiles y crea la tarea por (área, nivel) (`spec-gestion-excepciones.md` §4.2). Las ampliaciones se apilan append-only (regla 30-ter).

- *Cerrar y publicar.* Es una sola decisión en `ModalCurse` (regla 23) con dos compuertas: sin

excepciones sin justificar y Monto a Girar positivo, corte en \$1 (`giroCursable`, regla 13-septdecies). **Las dos compuertas no están al mismo nivel**: la del monto se re-comprueba en la mutación (`cerrarOferta`, con el comentario «la pantalla que apaga el botón no es el control (regla 24)»); la de las excepciones sin justificar vive **sólo en la pantalla** — `disabled= {sinComentario > 0 || !gOk}` del botón de `ModalCurse` y `if (!datosCurse) return;` en `ejecutarAccion`—, y `cerrarOferta` no consulta `excPend`, `sinComentario` ni el visado (§16). El CTA nombra lo que va a pasar: «Enviar a Comité y Publicar» cuando parte de la oferta no tiene cupo (`spec-ciclo-factura.md` §6). `cerrarOferta` escribe `patchCierre` (`ofertaCerrada`, `ofertaComunicada`, `negocioNum`, `publicacion: "electronica" | "fisica"`), invalida el cache del visado y, en la vía electrónica, llama a `enviarCierre` dentro del gesto del clic. Con eso los pendientes pasan de pronóstico (morado) a exigidos (rojo) (regla 54) y aparece el tab Verificación (regla 58).

- *Enviar a comité.* Lo genera el cierre: `asignarLineas` → `solicitudComiteDeOferta` (una

solicitud con N líneas de detalle `tipoLinea: "puntual"`, una por deudor sin cupo, `propFactoring = aprobada + pedido`) → `api1Inyeccion` → `repoSolicitudComite` y `deal.solicitudComite`; regla 15-bis (gates 106–107). Una solicitud idéntica no se re-inyecta (`mismaSolicitudComite`, regla 33); cruza de pestaña por `nex-solicitud` (regla 15-bis-bis). Lo que se **inyecta no depende del motivo**: `solicitudComiteDeOferta` arma siempre `tipo: "modificar"`, `subtipo: "agregar_deudores"`, `propGlobal: 0` y cada línea de detalle con `tipoLinea: "puntual"`; `RESOLUCION_COMITE` sólo aporta el rótulo `pide` / `alcance` que viaja en `detalle[]` (`par` / `lf4` → `puntual`; `lf1` → asignación por comité; `cliente` / `deudor` → global; `spec-ciclo-factura.md` §11), y `api1Inyeccion` inyecta ese objeto sin transformarlo: una factura rechazada por `cliente` o `deudor` produce igual una puntual cliente-deudor.

- *Bloqueo firme.* `visadoDealCalc` marca `rechFirme` sólo con rechazos no re-evaluables

(C30–C32 del cliente); `bloqueoFirmeInfo` arma la causa y el efecto de pérdida por bloqueo firme pasa la operación **entera** a `perdida` con `perdidaPor: "sistema"`. `retirarFacturaOferta` tiene tres llamadores y los tres son de verificación con motivo `"noConfirmada"`: ningún camino retira facturas por otorgamiento.

Diferencias y pendientes. M-28 implementado **bajo la lectura «no existir», a confirmar** (la solicitud sale al cerrar, no al simular; siempre líneas puntuales por deudor; publicar exige además Monto a Girar > 0), con un pendiente si el modelo quiere que el tipo de la solicitud siga al motivo (§16). Si el modelo quiso decir literalmente «existiendo suficiente línea» —p. ej. pedir la puntual igual para dejar constituida la línea del par —, el sistema NO lo hace: `solicitudComiteDeOferta` devuelve `null` salvo `ev.requiereComite > 0` y `cerrarOferta` sólo inyecta si hay solicitud, y la cláusula pasa a «implementado distinto». **M-19 implementado distinto:** la exigencia de justificación es de pantalla, no de la mutación; por el principio de la regla 24, que el propio cierre aplica al monto, falta la guarda en `cerrarOferta` (§16). **M-18 pendiente:** no existe una disposición bloqueante a nivel deudor (D02–D13 son excepciones visables no re-evaluables, `spec-gestion-excepciones.md` §2.4) ni un retiro automático de facturas por otorgamiento; implementarlo choca con la regla 33 (paquete cerrado sólo lectura) y con la regla 13 (después de aceptar, la única mutación es la verificación). Es la segunda contradicción de §15.

10. Aceptación formal del cliente y reapertura

Qué dice el modelo. No tiene cláusula propia de la firma: la nombra al publicar (M-19) y da por hecho que la oferta publicada se acepta. Lo que sigue es lo medido, para que el modelo lo adopte o corrija.

Qué hace el sistema hoy.

- *La firma es del cliente y sólo en el portal.* `enviarCierre` abre la pestaña del portal

en el gesto del clic, emite el OTP una vez (sólo el hash queda en el store) y envía el enlace; el correo es aviso sin enlaces. `confirmarCierre` recibe el `postMessage` de la firma, registra la evidencia del contrato por la vía electrónica (`registrarEvidenciaContrato`), calcula la oferta, recorta a `montoFirmado`, arma `dealFirmado` (`stage: "cesion"`, `clienteAcepto: true`, `reabierta: undefined`) **sólo como base en memoria** sobre la que se evalúan las compuertas (`visadoDeal`, `verifResumenDeal`, `evidenciaContratoOk`) y llama a `etapaTrasFirma` (§11). Lo que se **persiste** es el `patch` de `confirmarCierre`: `stage: stageFinal` — `"cesion"` sólo cuando `destino.integracion`, con `otorgada: true`, `integracion: "pendiente"`, `giroPendiente: true`, y `"otorgamiento"` con `giroPendiente: true` en cualquier otro caso— más `reabierta: undefined`; el patch **no** lleva `clienteAcepto`. El ejecutivo no puede mover una operación a Aceptada (regla 1). La firma cruza a la pestaña del tubo con `avisarTubo` (regla 55) y escribe el hilo «Cierre de negocio» a los aprobadores pendientes (regla 50).

- *Física vs electrónica.* O05 «Contrato firmado por cliente de la operación» (#305, Operaciones N3) existe siempre. Electrónica: la firma en el portal es la evidencia y O05 queda aprobado

sin visado. Física: O05 queda como excepción de Operaciones N3, el ejecutivo adjunta el comprobante y `aprobarExc` registra la evidencia al aprobarlo; revertir el visado revoca la evidencia. Entre las dos vías cambian 0 de 77 criterios (`spec-ciclo-factura.md` §16).

- *La evidencia es una huella.* `huellaOperacion`, cadena canónica + SHA-256; las

condiciones comerciales no entran (regla 23). `aprobacionFormalCliente` es la lectura: `clienteAcepto || otorgada || cierreFirmado || cesionExterna || stage ∈ {cesion, otorgamiento, giro}`, y **false** si `deal.reabierta`. **Vacío medido:** el único escritor persistido de `clienteAcepto: true` es el manejador del intent `cursar` del WhatsApp del cliente: cuando el cliente escribe que quiere cursar, la operación — todavía en `oferta` — queda con `clienteAcepto: true`, `cierreEnviado: true`, `ofertaComunicada: true` y status «Cliente aceptó · enlace de cierre enviado», y desde ahí `aprobacionFormalCliente` devuelve `true` sin firma en el portal. Es una aceptación por canal que la regla 1 declara inválida y que la lectura de la aprobación formal acepta (§16).

- *Reapertura.* `reabrirOperacion` comprueba `edicionOperacion` (regla 33), guarda

`enEdicion {desde, ts, por, versionAceptada, reserva}` y, si la edición revoca la firma, vuelve a `oferta` con `reabierto` puesta: el cliente debe firmar de nuevo. Sobrevive toda la evidencia (`repoVisado`, `repoVerifExc`, `repoVerifTel`, `repoNoConfirmadas`), las facturas no confirmadas quedan vetadas y la reserva sobrevive (regla 1). Girada o integrada al core no se reabre. Distinta de «Eliminar la simulación y vaciar la oferta» (regla 13-quaterdecies), que sólo aplica mientras la oferta sea del ejecutivo.

Diferencias y pendientes. Sin cláusula del modelo que contradecir. Tres vacíos del propio sistema:

`clienteAcepto` lo escribe el canal WhatsApp y no la firma, y `aprobacionFormalCliente` lo acepta (§16); el id `aceptadas` está en el catálogo pero la firma escribe `cesion` u `otorgamiento` (§2); y la inscripción de la cesión electrónica (AEC) entre la firma y la integración no tiene motor (`spec-ciclo-factura.md` §23b).

11. Después de la firma: otorgamiento, verificación, líneas del comité

Qué dice el modelo (M-20, M-21, M-23, M-25, M-29). Las excepciones y verificaciones ya resueltas no se vuelven a pedir. Si el deudor no pasa las reglas de verificación, todas las facturas de la oferta deben verificarse. Si el comité no aprueba la línea, se retiran las facturas de los deudores sin línea y se re-evalúa la operación, quedando cumplido el criterio de línea.

Qué hace el sistema hoy.

- *La bifurcación.* `etapaTrasFirma` recibe `requiereOtorg` (línea o deudores «Otro»),

`pendVisado` (OTG-02), `pendVerif` (VER-01) y `evidenciaOk` (GIR-02): si falta cualquiera → `otorgamiento` con su motivo («Otorgamiento / Verificación», una sola etapa); si no → `{stage: "cesion", integracion: "pendiente"}` («Pendiente Integración», sale del tubo). Regla 26, gate 88.

- *Otorgamiento post-firma.* Los apoderados deciden en la mesa o en el tab (`aprobarExc`, OTG-01

antes de escribir); el tab sobrevive a la firma. `otorgamientoCompleto` = en `otorgamiento`, sin bloqueo, con aprobación formal vigente, sin `excPend` ni `rechReev` (OTG-02 primero, regla 48) y con el atajo `otorgAuto` o todas las excepciones aprobadas. **No mira VER-01** por sí misma.

- *Verificación post-firma.* `verificarDeudor` registra las confirmadas en `repoVerifTel`

y retira las no confirmadas con `retirarFacturaOferta(id, fac, "noConfirmada")`: quedan vetadas en `repoNoConfirmadas`, y si la operación ya está aceptada se emite versión nueva con `recortarAsignacion` y la auditoría «el cupo liberado sigue reservado». La operación **encoge, no se pierde**; retirar la última factura es pérdida (`spec-ciclo-factura.md` §14). El veredicto se congela con el contacto (`congelarVeredicto`) y el predictor no vuelve a opinar.

- *Líneas del comité.* NEX sólo consulta (`api3EstadoProceso`, pull con «Consultar

estados», regla 15). El estado final simulado es «Aprobada» u «Observada» por `hash(idProceso) % 5`; Aprobada constituye la línea (`constituirLinea`: LF3 si puntual, LF2 si no) y se superpone al activo (`repoLineaComite`, regla 44). Una «Observada» deja las facturas `REQUIERE_COMITE` y LIN-01 frena la integración (`controlesIntegracion`). La única acción es manual, «Sacar facturas sin línea · N fact.» (en la pestaña de deudores del detalle, no en el menú Acciones; el estado vacío dice «No hay facturas por agregar ni facturas sin línea por sacar.»; comentario de la tarjeta del tubo), con la oferta abierta. `spec-asignacion-lineas.md` §8.7 la llama «Quitar lo que no tiene línea»: rótulo desfasado del fuente.

- *Salida de la etapa (desfase medido).* Dos escritores: `avanzarPipeline` movería

otorgamiento → cesion + integracion pendiente exigiendo `verifResumenDeal.pend === 0`, pero su `avanzarRef.current` **nunca se invoca**; y el `useEffect [deals, cfgVer]` con `deals.filter(otorgamientoCompleto)` pasa **directo** a `stage: "giro"`, «Girada · otorgada», sin Pendiente Integración, sin VER-01 y sin Operaciones N3 (`spec-ciclo-factura.md` §23c fila 4).

Diferencias y pendientes. M-20 y M-25 implementado distinto (§8). **M-23 decisión abierta:** la lectura «todas las facturas del deudor que falla» está implementada (regla 6); la lectura «todas las facturas de toda la oferta» sólo la hace la «Regla 0» de verificación (regla 6; `spec-verificacion-facturas.md` §4.0, cliente nuevo) y contradice la unidad por deudor. **M-29 pendiente:** no existe estado «Rechazada» del comité (buscado `rechazarSolicitud`, `resolverComite`, `decisionComite`: nada), ni retiro automático de las facturas sin línea, ni re-evaluación posterior; además choca con las reglas 13 y 33 (§15). Y el desfase del avance a Girada es del sistema, no del modelo: el camino vivo desde Otorgamiento se salta tres controles.

12. Giro

Qué dice el modelo (M-33, M-40). El motor de giros agrupa las facturas girables en express y/o normal según si el cliente es nuevo y los resultados de otorgamiento, verificación y líneas; el resultado de la evaluación son los montos a girar por giro normal o express.

Qué hace el sistema hoy.

- *GE / GN.* `asignarGiros` : GE / GN según el catálogo declarativo de tipos y sus cuatro

hechos por deudor (`spec-modelo-giro.md` §2, §2.1 y §4; regla 22, gates 81, 83–84). Acá sólo cuándo corre, qué recibe y qué devuelve (§7.4); el catálogo no se repite.

- *Inyección al core.* `aprobarIntegracion` : OTG-01 → `controlesIntegracion`

devuelve las faltas con código —OTG-02 (visado pendiente), VER-01 (`pend > 0`), LIN-01 (cada factura `CON_LINEA` desde la versión, falla cerrado sin asignación), GIR-02 («El paquete no es el autorizado», `evidenciaContratoOk` contra la huella)—; el botón se deshabilita con la lista y la compuerta se vuelve a llamar antes de escribir; el intento bloqueado va a auditoría (regla 41, gate 144). Si pasa: `repoGiro.set(id, giroEntregado)` congela el reparto y el patch es `{stage: "giro", integracion: "aprobada", giroPendiente: true}` = «Pendiente de Giro».

- *La noticia de Tesorería.* Girar no es una acción de NEX (regla 43): `moverEtapa` no ofrece `giro`

como transición manual, GIR-01 queda `aplicado: "externo"`. `recibirGiroTesoreria(ev, deal)` es pura: exige `operacionId`, `stage: "giro"` y `giroPendiente`; devuelve `{giroPendiente: false, giroTs, giroRef, status «Girada · ref», giroMonto sólo si el aviso lo trae}`; negativas `aviso_incompleto · no_inyectada · ya_girada` (gate 147; el 145 de la suite es ATR-01 y el 148 el congelado en la inyección; la regla 43 del vault todavía cita «caso 145», desfase de la renumeración del merge de la sesión paralela). El monto no se inventa.

Diferencias y pendientes. M-40 implementado. **M-33 implementado distinto:** el resultado de líneas no entra al criterio de giro (LIN-01 bloquea la integración aparte); dos tipos y no tres (`spec-modelo-giro.md` §7); GN como disyunción pendiente de confirmar (§2 «Supuesto explícito»). El contrato de entrega a Tesorería (payload, endpoint, idempotencia, evento) no está modelado (`spec-ciclo-factura.md` §23b).

13. Pérdida y sus causas

Qué dice el modelo (M-15, M-18). Una regla bloqueante fuera del límite impide avanzar; el modelo no define la pérdida como estado, la implica.

Qué hace el sistema hoy. La pérdida es terminal (regla 5): causa específica siempre (`causaPerdidaDeal` : `causaPerdida` → `bloqueoFirmeInfo.causa` → «Cesión externa» → «El cliente no aceptó la oferta» → «Sin contacto» → «Oportunidad perdida (causa no registrada)» **sólo si el status es el genérico** «no superó reglas de otorgamiento» → `deal.status || "Oportunidad perdida"` . Y como `STATUS_ETAPA` no tiene clave `perdida` , los dos escritores manuales escriben `status: STATUS_ETAPA["perdida"] || d.status` = el status de la etapa anterior (`moverEtapa` , `moveTo`): para una pérdida por arrastre o por «Avanzar», `causaPerdidaDeal` devuelve como «causa» el status de la etapa de origen —p. ej. «En oferta y negociación»—, no «causa no registrada», etapa de origen, actor, cierre de tareas en cascada, badges accionables suprimidos; reabrir es una operación nueva con referencia. Ocho escritores de `stage: "perdida"` (seis con causa y dos manuales sin ella):

ESCRITOR	CAUSA	ACTOR	ESTADO
<code>reject(id, closeReason, extra)</code>	Manual desde el detalle; deriva <code>bloqueo_firme</code> si hay <code>bloqueoFirmeInfo</code> , si no <code>cliente_declino</code> ; guarda <code>cedidaCompetidor</code> / <code>tasaCierreCompetidor</code>	Ejecutivo	vivo
<code>evaluarPerdidas</code> (cron)	Cesión al competidor por AEC, <code>rndDetBool</code> 12% una vez por oportunidad	Sistema	vivo
<code>evaluarPerdidas</code> (cron)	Oferta no aceptada si <code>d.perdedor</code> , una sola vez en Oferta, respetando <code>negocioNum</code> / demo / <code>fueraAtribucion</code>	Sistema	vivo
<code>useEffect</code>	Bloqueo firme de otorgamiento (<code>rechFirme</code> o excepción rechazada por un apoderado): <code>perdidaOtog</code> , <code>motivoPerdida: "bloqueo_firme"</code> , <code>perdidaPor: "sistema"</code>	Sistema	vivo
<code>avanzarPipeline</code>	Cesión a la competencia documento a documento (sólo si todas las facturas están cedidas); «el cliente no aceptó» por <code>d.perdedor</code>	Sistema	sin llamador
<code>moverEtapa(id, stageId)</code>	Botón «Avanzar» del detalle (<code>onMover(deal.id, avanzarA)</code> ; <code>avanzarA</code> = siguiente id de <code>STAGE_ORDER</code> salvo <code>aceptadas</code>). Escribe <code>{...d, stage: stageId, status: STATUS_ETAPA[stageId]}</code> para cualquier etapa salvo <code>aceptadas</code> y <code>giro</code> ; <code>cesion</code> sólo pasa OTG-02 y entra <code>sin integracion</code> ; <code>otorgamiento</code> , <code>perdida</code> (sin <code>causaPerdida</code> ni <code>etapaPerdida</code> , contra la regla 5, y con <code>status</code> de la etapa de origen porque <code>STATUS_ETAPA</code> no tiene <code>perdida</code>) y volver a <code>oferta</code> / <code>prospeccion</code> no tienen guarda	Ejecutivo	vivo
<code>moveTo(stageId)</code>	Arrastre del Kanban (<code>onDrop(stage.id)</code>). Escribe <code>stage: stageId</code> bloqueando sólo <code>aceptadas</code> , la salida de <code>perdida</code> y el retroceso desde <code>aceptadas</code> / <code>cesion</code> / <code>giro</code> ; a <code>perdida</code> entra sin causa y con <code>status: STATUS_ETAPA["perdida"] d.status</code> = el de la etapa de origen	Ejecutivo	vivo

Dos puertas de negocio a la pérdida por otorgamiento (`spec-ciclo-factura.md` §13): el rechazo firme (C30–C32 del cliente, nunca del deudor) y la excepción rechazada por un apoderado con atribución (no hay apelación dentro de la operación). El bloqueo firme tiene precedencia en la causa. Un deudor que no confirma **no** es pérdida: es encogimiento (§11), salvo que retire la última factura.

Diferencias y pendientes. M-15 implementado distinto: el modelo define «bloqueante» como una regla que detiene la operación hasta que se retiran las facturas del deudor (M-18: a nivel deudor y reversible); el sistema tiene el rechazo firme sólo en tres reglas del **cliente** (C30–C32, `tHard`, rótulo «bloqueo firme») y su efecto no es «impedir avanzar» sino **pérdida automática y terminal** de la operación entera (`useEffect` escribe `stage: "perdida"` con `perdidaPor: "sistema"` para toda operación activa con `bloqueoFirmeInfo`; `snapVersionCli` `estado = nRechFirme ? "rechazada" ...`; reglas 4 y 5), sin vía de destrabe. M-18 pendiente (§9): no hay pérdida ni retiro a nivel deudor porque no hay bloqueo a nivel deudor. La pérdida por cesión externa vive en dos sitios, uno vivo (cron, sintético al 12%) y uno sin llamador (documento a documento por A2): decidir cuál es el camino de producción es parte de la limpieza del inbound, no de este modelo.

Parte III · Invariantes del proceso

Las reglas del vault que gobiernan el proceso, con su número tal cual está en `vault/conocimiento/reglas/<tema>.md` y el gate según `vault/conocimiento/invariantes.md`. Las reglas propias de cada motor que el cuerpo cita (18, 20, 21, 30-ter, 44, 45, 53) viven en `vault/conocimiento/reglas/<tema>.md`; el motor que gobiernan las detalla en su spec. La columna Gate sigue al índice, cotejado caso por caso contra `tests_asignacion_lineas.js` (regla núcleo 2): el número de un caso nombra lo que prueba.

REGLA	TEMA	ENUNCIADO (RESUMEN FIEL)	GATE
1	curse_firma_y_etapas	Curse nunca por email: el cliente acepta sólo firmando en el portal; el ejecutivo no puede mover a Aceptada; reabrir revoca la firma; girada o integrada no se reabre; al reabrir se conservan los repositorios y las no confirmadas quedan vetadas	~24-26
5	curse_firma_y_etapas	Pérdida es estado terminal: causa específica, etapa de origen, actor, cierre de tareas en cascada; reapertura = operación nueva con referencia	117, 118, regla_5.test.mjs
12-bis	curse_firma_y_etapas	Lo que saca una oportunidad de Prospección es la simulación, no editar el paquete; invariante dual prospección/oferta	e2e-12-bis-a a e, regla_12_bis.test.mjs
23	curse_firma_y_etapas	Publicar se decide en <code>ModalCurse</code> , dos vías (electrónica / física); O05 existe siempre; la evidencia es la huella del paquete sin condiciones comerciales	~85
24	curse_firma_y_etapas	GIR-02: <code>evidenciaContratoOk</code> devuelve el porqué; el destino apagado se muestra con motivo; <code>aprobarIntegracion</code> lo comprueba antes de escribir	~86
26	curse_firma_y_etapas	Máquina post-firma: «Otorgamiento / Verificación» → «Pendiente Integración» → «Pendiente de Giro»; <code>estadoOperacion</code> único traductor; <code>etapaTrasFirma</code> pura	88
28	curse_firma_y_etapas	Nombre y color de cada etapa son del tenant; el código no se edita; «Oferta publicada» no es un stage; <code>etapaDeDeal</code> única puerta al rótulo	110-111

REGLA	TEMA	ENUNCIADO (RESUMEN FIEL)	GATE
30	curse_firma_y_etapas	El menú Acciones no cierra ni avanza: cerrar tiene un solo camino (<code>ModalCurse</code> con sus dos compuertas)	e2e-30, regla_30.test.mjs
33	curse_firma_y_etapas	El paquete cerrado es sólo lectura (salvo <code>noConfirmada</code>); cerrada, publicada y reabierto son tres hechos; re-cerrar conserva lo excepcionado y no re-inyecta una solicitud idéntica	140, regla_33.test.mjs
41	curse_firma_y_etapas	<code>controlesIntegracion</code> puro: OTG-02, VER-01, LIN-01 factura por factura desde la versión, GIR-02; se vuelve a llamar antes de escribir	144, regla_41.test.mjs
43	curse_firma_y_etapas	Girar no es acción de NEX: termina en <code>aprobarIntegracion</code> ; el giro vuelve como callback (<code>recibirGiroTesoreria</code> pura); la asignación se congela en la inyección	147, 148, regla_transiciones.test.mjs
50	otorgamiento_y_atribucion	Al firmar el cliente, el sistema escribe el hilo «Cierre de negocio» a ejecutivo y aprobadores pendientes	154, regla_aviso_cierre.test.mjs
55	curse_firma_y_etapas	La firma cruza a la pestaña del tubo (<code>avisarTubo</code> antes de aplicar el patch)	regla_estado_pestanas.test.mjs
58	curse_firma_y_etapas	La oferta publicada es un estado (<code>ETAPA_PUBLICADA</code>); <code>patchCierre</code> asienta <code>ofertaComunicada</code> ; el tab Verificación aparece al cerrar	e2e-58, regla_58.test.mjs
54	ui_detalle_y_tubo	Pendientes de otorgamiento y verificación: pronóstico (morado) mientras se simula, exigidos (rojo) al publicar	158, regla_54.test.mjs
8	oferta_pricing_y_giro	«Cerrar oferta» es prerequisite de publicar; el Agente IA es opcional; el piso de la operación es el del deudor más exigente	119, 149

REGLA	TEMA	ENUNCIADO (RESUMEN FIEL)	GATE
13- quaterdecies	oferta_pricing_y_giro	«Eliminar la simulación y vaciar la oferta» devuelve al estado de entrada sin tocar la evidencia; sólo mientras la oferta sea del ejecutivo	e2e-13-quaterdecies, regla_13_quaterdecies.test.mjs
13-sexdecies	oferta_pricing_y_giro	Armar la oferta a mano tiene salida «Simular la oferta»; la oferta vacía es un estado normal	e2e-13-sexdecies-a a d, regla_13_sexdecies.test.mjs
13- septdecies	oferta_pricing_y_giro	Monto a Girar no positivo se simula pero no se cursa: giroCursable puro, corte en \$1	108
14	oferta_pricing_y_giro	Reevaluación explícita: editar la oferta no dispara cálculo; «Por evaluar» hasta Re-evaluar; verificación y líneas en la misma pasada	124, e2e-14-a/b/c, regla_14.test.mjs
22	oferta_pricing_y_giro	asignarGiros : suma por tipo = monto a girar; GE = verificado y sin marcas; por deudor; se congela (la regla 22 dice «al aceptar», repoGiro ; la 43 lo movió a la inyección en aprobarIntegracion); el id no cambia con el cierre del día	81, 83–84, ~78–80, ~82
4	otorgamiento_y_atribucion	Motor de otorgamiento = Modelo de Riesgo v1.0: 77 reglas; piso por monto; ruteo (área, nivel); KNOCKOUT C30/C31/C32 → pérdida automática	44, 46, 48, 56–59, ~38–41, ~43, ~45, ~47, ~49–51
35	otorgamiento_y_atribucion	Una regla excepcionable sin aprobador posible no se ejecuta (no_ejecutada), nunca en silencio	141, 143, regla_35.test.mjs
48	otorgamiento_y_atribucion	El atajo del otorgamiento automático no pasa por encima de OTG-02 (otorgAutoVigente)	153, regla_48.test.mjs
6	verificacion	Verificación aislada y por deudor; dos segmentos; «Regla 0» (primera operación del cliente, viñeta de la regla	~27–32, ~52–55, ~76–77

REGLA	TEMA	ENUNCIADO (RESUMEN FIEL)	GATE
		6); V01 compuerta; unanimidad; el veredicto se congela con el contacto; confirmación parcial	
7	lineas_y_solicitud_comite	asignarLineas pura; cinco líneas; cabe en los tres niveles; recálculo siempre completo	~1–15, ~89
12	lineas_y_solicitud_comite	La reserva no es de NEX: el sistema de líneas reserva al firmar, el core commitea al aprobar; requiere_resimulacion si el curse reevalúa distinto	122, regla_12.test.mjs
13	lineas_y_solicitud_comite	Cada simulación emite una versión append-only; una aceptada se lee de su versión; después de aceptar sólo encoge	~16–23
15	lineas_y_solicitud_comite	NEX sólo inyecta (API 1) y consulta (API 2/3, pull); resuelve el sistema externo	125, e2e-15, regla_15.test.mjs
15-bis	lineas_y_solicitud_comite	La solicitud se genera sola al cerrar: una solicitud con N líneas puntuales; lo pedido se suma a la vigente	106–107
15-bis-bis	lineas_y_solicitud_comite	La solicitud inyectada cruza de pestaña; idempotente por la solicitud; la inyección no exige la firma	126, 146, e2e-15-bis-bis-a/b, regla_15_bis_bis.test.mjs
OTG-01	contrato con el servidor	Sólo aprueba quien tiene atribución; el resolver recalcula desde el rol del token	135, regla_otg_01.test.mjs
OTG-02	contrato con el servidor	No avanza a Cesión con excepciones o rechazos re-evaluables sin resolver	88, regla_transiciones.test.mjs
VER-01	contrato con el servidor	No cursa con verificación pendiente	52, 88
LIN-01	contrato con el servidor	La operación cabe en la línea disponible del cliente al armarla (contrato); que en la integración se mire factura por factura desde la versión es la regla 41 (caso 144)	134

REGLA	TEMA	ENUNCIADO (RESUMEN FIEL)	GATE
GIR-01	contrato con el servidor	No gira sin pasar por Cesión; NEX no lo aplica (externo)	136, regla_transiciones.test.mjs
GIR-02	contrato con el servidor	El paquete girado es el que se autorizó (huella contra huella)	86, 88
ATR-01	contrato con el servidor	El descuento no excede la atribución del rol sin la jefatura	137, 143, regla_atr_01.test.mjs

Parte IV · Conciliación del modelo con el sistema

14. Cláusula por cláusula

Estados: **implementado** · **implementado distinto** (con otra forma o alcance, y se dice cuál) · **pendiente** (no existe) · **decisión abierta** (el modelo y una regla vigente se contradicen). La evidencia es la **definición**: la regla del vault por su número, la sección del spec, o la condición del fuente por su nombre —el símbolo que la encarna—, medida al 22-09-2026. Nunca una línea de código: la línea cambia con el próximo commit; la definición, no.

ID	CLÁUSULA	ESTADO	EVIDENCIA
M-01	Las facturas llegan por DTESync con cesiones, NC, aceptaciones y reclamos	implementado distinto	<code>facturaDeDTE</code> (reclamo y NC como banderas de <code>EstadoDTE</code>); cesiones por A2 (<code>spec-ciclo-factura.md</code> §1); «Aceptaciones» no existe en el A1 ni en ningún spec; en el fuente sólo como dato sintético sin consumidor (<code>facturasDeCandidata</code> : "Aceptada" "Reclamada" "Sin acuse")
M-02	Cada hora corren los motores de inbound	implementado distinto	La corrida horaria es una simulación en el navegador : <code>setInterval(..., CRON_MS)</code> sólo si <code>CFG_ACTIVADA.modosDemo !== false</code> (su comentario: sin modo demo no corre) → <code>tickCron</code> → <code>correrProceso</code> ; <code>CRON_MS = cfgT.cronMs</code> (<code>cronMs: 3500</code>). <code>frecuenciaMin: 60</code> es un parámetro huérfano : se edita en Configuración > Operación y ningún cron lo lee, como <code>horaInicio</code> / <code>horaFin</code> . El job real del backend está pendiente (<code>spec-inbound-facturas.md</code> §10.4)
M-03	Al detectar facturas se abre una oportunidad al cliente	implementado	<code>correrProceso</code> agrupa por cedente; tope <code>MAX_NUEVOS = 40</code> ; <code>spec-inbound-facturas.md</code> §6
M-04	Si ya existe una oportunidad abierta se actualiza	implementado	<code>aplicar(warn)</code> : suma a <code>facturasDisponibles</code> deduplicando; la oferta no se toca; aceptada/cursada/perdida → otra
M-05	Cuantifica facturas segmentadas en Prime y Otros	implementado distinto	<code>capacidadDeudores</code> tiene un solo llamador, la tarjeta del tubo al dibujarse (lookup sobre A23, «no una corrida del motor», dice su comentario): se calcula en el render, no se persiste en la oportunidad ni la produce el inbound (<code>correrProceso</code> no la escribe); en producción no habría dónde consultarlo fuera de la pantalla. <code>chipTramo</code> ; Prime = listas (<code>CRITERIO_PRED</code>)
M-06	Join con líneas: con línea, Prime con línea, otros sin línea	implementado distinto	<code>capacidadDeudores</code> : <code>primeConLinea · otrosConLinea · sinLinea</code> , cota superior (lo declara el tooltip del chip); se calcula en el render de la tarjeta, no en la oportunidad (M-05); <code>lineaCreditoDe</code>
M-07	Hora de corte 23:00 elimina las no gestionadas	implementado distinto	Sin hora de reloj en la lógica (<code>horaInicio</code> / <code>horaFin</code> son parámetro declarado sin efecto); <code>corridas % HORAS_DIA === 0</code> → <code>rolloverDia</code> reabre con el mismo id; <code>reaperturaDiaria</code> / <code>etapaNoGestionada</code>
M-08	A las 06:00 se reinicia el proceso	pendiente (parámetro huérfano)	Buscado <code>06:00</code> , hora de reinicio: nada. <code>CFG_OPER_BASE.horaInicio = "08:00"</code> / <code>horaFin = "18:00"</code> («ventana horaria de operación (inbound + actualizaciones)») se editan en Configuración > Operación y ningún job los lee: el cron es <code>setInterval(..., CRON_MS)</code> y el cierre es <code>corridas % HORAS_DIA === 0</code> ; <code>rolloverDia</code> reabre en el mismo tick del cierre
M-09	Candidata si no cedida, no reclamada, sin NC	implementado distinto	el criterio «Buena factura» de <code>CRITERIO_PRED</code> no consulta <code>cedida</code> ; <code>estadoCandidata</code> la bloquea al incorporar; exige

ID	CLÁUSULA	ESTADO	EVIDENCIA
			además <code>credito</code>
M-10	Reglas por RUT emisor, fecha de emisión, reclamo, NC, cesiones previas; lista de emisores con tags	implementado distinto	<code>INBOUND_RULES</code> + <code>CRITERIO_PRED</code> ; sin criterio por fecha ni cesión previa; reclamo y NC sólo dentro de «Buena factura», no seleccionables por sí solos; tags del deudor, no del emisor; criterio desconocido califica todo
M-11	El ejecutivo toma la oportunidad sin oferta y selecciona	implementado	<code>facturasOp: []</code> al nacer; <code>estadoCandidata</code> ; <code>incorporarFacturasOferta</code> / <code>retirarFacturaOferta</code> (regla 33); regla 13-sexdecies
M-12	Cada cambio de selección re-evalúa	decisión abierta	Regla 14 (gate 124, e2e-14): la re-evaluación se pide; <code>reevaluarLinea</code> ; <code>spec-otorgamiento.md</code> §2, <code>spec-asignacion-lineas.md</code> §8.4, <code>spec-ciclo-factura.md</code> §3
M-13	Cinco motores en paralelo, independientes	implementado distinto	Independientes (<code>spec-ciclo-factura.md</code> §0 punto 1) pero tres gestos de evaluación (<code>reevaluarCliente</code> ; <code>spec-gestion-excepciones.md</code> §4.1, §5.5); pricing en el render
M-14	Otorgamiento evalúa por empresa, no por factura	implementado	<code>evaluarOtorgItems</code> , <code>stKey</code> ; <code>deudorBlock</code> ; regla 4
M-15	Reglas bloqueantes impiden avanzar	implementado distinto	El modelo define «bloqueante» como lo que detiene la operación hasta retirar las facturas del deudor (M-18: por deudor y reversible); el sistema tiene rechazo firme sólo en C30–C32 del cliente (<code>tHard</code> , «bloqueo firme»; <code>visadoDealCalc</code> <code>rechFirme</code>) y su efecto es pérdida automática y terminal de la operación entera (<code>useEffect</code> , <code>perdidaPor: "sistema"</code> ; <code>snapVersionCli</code> <code>estado = "rechazada"</code> ; reglas 4 y 5), sin vía de destrabe
M-16	Reglas re-evaluables	implementado	<code>NO_REEV_CLIENTE</code> es la lista de las no re-evaluables (<code>reglaReev</code>), vigente —la IIFE del Modelo de Riesgo la rellena — con los 28 de <code>spec-gestion-excepciones.md</code> §2.4 (el literal inicial está muerto); <code>rechReev</code> es su complemento → sujeta; <code>reevaluarCliente</code> emite versión
M-17	Reglas excepcionables por apoderado según criticidad	implementado	Ruteo (área, nivel) + piso <code>PISO_ATRIB_MONTO</code> / <code>CFG_TRAMOS</code> (regla 4); <code>puedeAprobarExc</code> (regla 18); <code>aprobarExc</code> con OTG-01
M-18	Deudor bloqueado → el sistema retira sus facturas automáticamente	pendiente	<code>retirarFacturaOferta</code> : tres llamadores, todos <code>"noConfirmada"</code> ; el bloqueo firme pierde la operación entera; no hay knockout de deudor (<code>spec-gestion-excepciones.md</code> §2.4)
M-19	Justificar cada excepcionable antes de enviar a comité y publicar	implementado distinto	La justificación la exige la pantalla del tab, no <code>solicitarAprobacionExc</code> (guarda sin validar); la Pre-evaluación solicita todas las pendientes sin justificar; <code>cerrarOferta</code> re-comprueba sólo el monto (<code>giroCursable</code>) y la compuerta de excepciones vive sólo en

ID	CLÁUSULA	ESTADO	EVIDENCIA
			<code>ModalCurse</code> (regla 30): falta la guarda en la mutación (regla 24); CTA «Enviar a Comité y Publicar»
M-20	Excepción resuelta no se vuelve a pedir; versionadas	implementado distinto	<code>reevaluarCliente</code> ; <code>repoVisado</code> por <code>dealId + stKey</code> , <code>VISADO_STATE</code> ; sobrevive por clave estable, no dentro de la versión
M-21	Si ya no era necesaria, se marca cumple y se elimina	implementado distinto	<code>visadoDealCalc</code> consulta sólo lo que gatilla hoy; la entrada en <code>repoVisado</code> , <code>SOLICITUD_EXC</code> , el hilo y la tarea quedan huérfanos
M-22	Verificación por empresa para decidir el contacto	decisión abierta	Modelo: «empresa emisora» (= cedente); sistema: por deudor (<code>verifDecision</code> ; <code>verifFactura</code> evalúa todas las facturas del deudor); regla 6; <code>spec-verificacion-facturas.md</code> §21 («La decisión es por deudor dentro de la operación, no por factura»; «empresa emisora» no aparece en ese spec: es el texto de M-22 y contradice a ambos). §15 #5
M-22-bis	El contacto obtiene si la factura es verídica, si los bienes/servicios se prestaron según lo pactado y confirma la fecha de pago (cláusula del motor b del modelo, agregada en esta versión)	implementado	Por factura, en <code>repoVerifTel</code> ; no versionado, clave estable como M-25. <code>DrawerVerificacion</code> lleva el checklist <code>{existencia, recepcion, fechaPago}</code> (rótulos «Existencia de la factura» / «Recepción conforme» / «Fecha de pago»; mesa <code>CHECKS</code>) y exige los tres checks más la fecha comprometida antes de confirmar (<code>completo</code>); <code>verificarDeudor</code> y <code>confirmarLlamadaTel</code> de <code>VerificacionTab</code> persisten <code>{por, fecha, checklist, contacto, compromiso, respaldo, sinRespaldo, notas}</code> ; <code>verifFactura</code> lo lee (<code>checks: [existencia, recepcion, fechaPago]</code>); regla 6 («checklist existencia/recepción/fecha»). El veredicto congelado sigue siendo por deudor (<code>VERIF_VEREDICTO</code>). Pendiente sólo documental: <code>spec-verificacion-facturas.md</code> no describe el checklist (§21: el detalle por factura «no es responsabilidad de esta función»)
M-23	Si el deudor no pasa, todas las facturas de la oferta se verifican	decisión abierta	<code>verifFactura</code> (todas las del deudor); «Regla 0» de verificación (regla 6; <code>spec-verificacion-facturas.md</code> §4.0: todas las de la oferta sólo para cliente nuevo)
M-24	La verificación corre en cada evaluación	implementado distinto	Corre en el render del detalle (<code>verifFactura</code>) y en la versión (<code>snapVersionCli.verificacion</code> , sólo al hacer «Re-evaluación de la simulación»); ningún evento de evaluación la invoca: <code>simularOferta</code> no llama a <code>verifDecision</code> y «Re-evaluar operación» (<code>reevaluarLinea</code>) sólo apaga <code>reevalPend</code> tras 700 ms; regla 14 (gate 124); tras el contacto se congela
M-25	Factura verificada no se vuelve a pedir; versionadas	implementado distinto	<code>repoVerifTel</code> por (deal, factura); <code>congelarVeredicto</code> ; <code>limpiarSimulacion</code> no los toca; clave estable, no versión

ID	CLÁUSULA	ESTADO	EVIDENCIA
M-26	Líneas en cada evaluación, cada factura con línea asignada	implementado distinto	<code>asignarLineas</code> (<code>CON_LINEA</code> / <code>REQUIERE_COMITE</code>) corre en el render del detalle (<code>evalLin</code>) y en la versión (<code>snapVersionCli</code> , sólo desde <code>reevaluarCliente</code> y <code>retirarFacturaOferta</code>); la simulación no lo ejecuta (<code>simularOferta</code> sólo escribe <code>simulado</code> , <code>stage</code> y <code>finanzasDe</code>) ni «Re-evaluar operación» (<code>reevaluarLinea</code>); LIN-01 en <code>controlesIntegracion</code>
M-27	Cuatro líneas LF1–LF4	implementado distinto	El modelo dice cuatro; el sistema asigna contra cinco : LF1–LF4 (cascada por estado del cliente, <code>spec-asignacion-lineas.md</code> §3.5; <code>asignarLineas</code>) más la línea global del deudor como tercer nivel obligatorio —una factura cursa sólo si <code>m ≤ restPar && m ≤ dispCliente && m ≤ restDeudor</code> ; <code>capacidadDeudores</code> «Nivel 3: la línea GLOBAL del deudor»— (regla 7 «cinco líneas»; origen A23 bloque <code>LINEA_DEUDOR</code> , regla 44; y la línea del RUT cliente ES la suma de sus líneas, regla 45, ADR-0011). El modelo debe decir si la línea global del deudor es parte del modelo de líneas o un control aparte (§16)
M-28	Al publicar, solicitud automática al comité	implementado (a confirmar)	<code>cerrarOferta</code> → <code>solicitudComiteDeOferta</code> (siempre <code>tipoLinea: "puntual"</code> , <code>propGlobal: 0</code> ; el motivo sólo rotula) → <code>api1Inyeccion</code> ; regla 15-bis. Estado bajo la lectura «no existir», a confirmar por el usuario (§9); si el modelo quiso decir literalmente «existiendo suficiente línea», el sistema NO lo hace — <code>solicitudComiteDeOferta</code> devuelve <code>null</code> salvo <code>ev.requiereComite > 0</code> y <code>cerrarOferta</code> sólo inyecta si hay solicitud— y la cláusula pasa a «implementado distinto»
M-29	Comité no aprueba → retirar facturas sin línea y re-evaluar	pendiente	<code>api3EstadoProceso</code> : sólo «Aprobada» / «Observada»; sin manejador de rechazo; choca con reglas 13 y 33
M-30	La línea es un monto; la asignación es por factura	implementado	<code>asignarLineas</code> <code>resFacturas</code> con su <code>origen</code> ; regla núcleo 9; <code>spec-asignacion-lineas.md</code> §21, §2.4
M-31	Una factura puede asociarse a más de una línea (LF2–LF3)	implementado	<code>origen.push</code> ; «una factura se reparte entre ambas», dice el comentario de <code>asignarLineas</code> ; sólo dentro de la cascada del par
M-32	No se cursa con línea por monto parcial	implementado	La condición de <code>asignarLineas</code> : <code>m ≤ restPar</code> y <code>≤ dispCliente</code> y <code>≤ restDeudor</code> ; si no, completa a <code>REQUIERE_COMITE</code> ; LIN-01 (regla 41)
M-33	Giro GE / GN según cliente nuevo, otorgamiento, verificación y líneas	implementado distinto	<code>asignarGiros</code> : líneas no entran al criterio; dos tipos; GN disyunción (<code>spec-modelo-giro.md</code> §2)

ID	CLÁUSULA	ESTADO	EVIDENCIA
M-34	Pricing asigna tasa por deudor	implementado	<code>spreadSugerido</code> ; <code>SPREAD_MIN_DEUDOR</code> ; <code>tasaDe</code> ; <code>prorratearOperacion</code>
M-35	Tasa equivalente vs. tasa de los últimos negocios	implementado distinto	<code>tasaEquivalente</code> ; <code>tasaUltimoNegocio</code> (el último negocio del cliente , sintético y sembrado sólo por cliente en <code>historialComercial</code> , no del par); <code>tasaModo</code>
M-36	Condiciones comerciales versionadas	implementado distinto	<code>snapVersionCli</code> y <code>huellaOperacion</code> no contienen tasa, comisión ni anticipo; la simulación inicial no emite versión y la v1 se emite retroactiva en la primera re-evaluación
M-37	Resultado: requisitos de excepción	implementado	<code>evaluarOtorgItems</code> ; tab Otorgamiento y mesa; regla 54
M-38	Resultado: requisitos de verificación	implementado	<code>verifResumenDeal</code> ; tab y mesa <code>VerificacionView</code> (regla 53)
M-39	Resultado: asignación de líneas y solicitudes	implementado	<code>asignarLineas</code> devuelve <code>solicitudes</code> ; <code>lineaDeVersion</code> ; <code>solicitudComiteDeOferta</code>
M-40	Resultado: montos a girar GE / GN	implementado	<code>asignarGiros</code> <code>porTipo</code> / <code>porDeudor</code> ; <code>giroDeal</code> / <code>giroCongelado</code> ; regla 22

Conteo sobre las 40 cláusulas: 15 implementadas (M-28 bajo la lectura «no existir», a confirmar) · 19 implementadas distinto (M-02, M-15, M-24 y M-26 pasaron a esta columna en la revisión contra el fuente: la corrida horaria es simulación con parámetro huérfano, el bloqueo firme es pérdida terminal y no freno, y ni la simulación ni «Re-evaluar operación» invocan los motores) · 3 pendientes (M-08 es de calendario, con parámetro huérfano; M-18 y M-29 son de máquina de estados) · 3 decisiones abiertas (M-12, M-22, M-23); más M-22-bis, agregada en esta versión e **implementada** (por factura, clave estable; pendiente sólo documentarla en el spec de verificación). Las «implementadas distinto» no son defectos por sí solas: cada una tiene una lectura del modelo que el sistema no adoptó, y va a §15 o §16.

15. Las contradicciones que hay que decidir

Cada una con las dos lecturas. Ninguna se cierra programando: se cierra con un ADR que reemplace la regla vigente o con una corrección del modelo.

#	TEMA	LECTURA A (MODELO)	LECTURA B (VIGENTE, CON EVIDENCIA)	QUÉ HAY QUE DECIDIR
1	Re-evaluación al cambiar la selección (M-12)	Cada vez que el ejecutivo selecciona o modifica su selección, la oferta se evalúa o re-evalúa	La re-evaluación se pide y cada evaluación deja evidencia; editar sólo actualiza lo aritmético (regla 14, gate 124, e2e-14-a/b/c; <code>spec-otorgamiento.md</code> §2; <code>spec-asignacion-lineas.md</code> §8.4; <code>spec-ciclo-factura.md</code> §3; <code>reevaluarLinea</code>)	Si cada clic emite versión y consume la API de líneas A23, o sólo el gesto explícito; reemplazar la regla 14 por ADR
2	Deudor bloqueado → retiro automático de sus facturas (M-18)	Existe un bloqueo a nivel deudor y el sistema retira sus facturas como resultado de la simulación	Los knockouts son C30–C32 del cliente (<code>spec-ciclo-factura.md</code> §13); un rechazo firme pierde la operación entera (reglas 4 y 5; <code>useEffect</code>); D02–D13 son excepciones no re-evaluables (<code>spec-gestion-excepciones.md</code> §2.4); paquete cerrado sólo lectura (regla 33); <code>retirarFacturaOferta</code> sólo por <code>noConfirmada</code>	Si existe la disposición «bloqueante del deudor»; retiro automático o del ejecutivo; en qué momento (simulación, cierre, después de la firma)
3	Cierre del día: eliminar vs reabrir (M-07, M-08)	A la hora de corte se eliminan las oportunidades no gestionadas y al día siguiente se reinicia	<code>rolloverDia</code> reabre con el mismo id y el paquete actualizado (regla 22); el corte es por conteo de corridas. <code>spec-ciclo-factura.md</code> §17 dice que el rollover «cierra y re-origina las oportunidades del inbound que nadie gestionó» y que «Es una oportunidad nueva , con su propio identificador»: el vault y el fuente («EL ID NO CAMBIA») contradicen a ese spec	Hora de reloj o conteo; id nuevo o el mismo; qué se conserva (trazas, ejecutivo, contactos); qué es «no gestionada» (sólo Prospección u Oferta sin publicar); hora de corte y de reinicio como parámetros que el job consuma —la ventana <code>horaInicio</code> / <code>horaFin</code> ya existe en <code>CFG_OPER_BASE</code> y nadie la lee— y qué hace el job con el stream entre corte y reinicio
4	Comité que no aprueba (M-29)	Se retiran las facturas de los deudores sin línea y se re-evalúa; el criterio de línea queda cumplido	Después de aceptar la operación sólo encoge y la única mutación es el retiro por verificación (regla 13; <code>spec-asignacion-lineas.md</code> §4.3); paquete cerrado sólo lectura y reabrir revoca la firma (reglas 33 y 1); el comité simulado nunca rechaza (<code>api3EstadoProceso</code>)	Un estado «Rechazada» por línea de detalle; retiro automático o del ejecutivo; antes o después de la firma (segunda mutación admitida, con versión y motivo); qué pasa si la operación queda en cero
5	Unidad de la verificación (M-22, M-23)	Las reglas corren a nivel de la empresa emisora y, si falla, todas las facturas de la	La unidad es el deudor: V01–V10 son atributos del deudor o del par (regla 6; <code>spec-verificacion-facturas.md</code> §2.1 «La decisión es por deudor dentro de la operación» —«empresa emisora» no aparece en ese spec, es el texto de M-22—; <code>verifDecision</code> ;	Si «empresa emisora» quiso decir deudor; si «todas las facturas de la oferta» son las del deudor que falla o las de todos

#	TEMA	LECTURA A (MODELO)	LECTURA B (VIGENTE, CON EVIDENCIA)	QUÉ HAY QUE DECIDIR
		oferta se verifican	<code>verifFactura</code>); sólo la «Regla 0» de verificación (regla 6; <code>spec-verificacion-facturas.md</code> §4.0) verifica toda la oferta	
6	Cedida como exclusión del inbound (M-09)	Candidata sólo si no está cedida a otro factoring	El inbound no excluye la cedida (<code>CRITERIO_PRED</code>); bloquea al incorporar (<code>estadoCandidata</code>); declarado desfase con el PDF de política (<code>spec-inbound-facturas.md</code> §3, §11 fila 1, §12.1; <code>spec-ciclo-factura.md</code> §23c fila 6)	Agregar la cesión como cuarta condición del filtro; hoy infla el dimensionamiento

16. Lo pendiente, por motor

Cada ítem dice qué pide el modelo o el spec, qué se buscó en el fuente y qué se encontró.

MOTOR	PENDIENTE O DECISIÓN ABIERTA	QUÉ SE BUSCÓ Y QUÉ SE ENCONTRÓ
Inbound	Hora de corte y de reinicio (M-07, M-08) y frecuencia de la corrida (M-02) como parámetros del tenant que el job del backend consuma (<code>spec-inbound-facturas.md</code> §10.4); hoy la ventana y la frecuencia existen como parámetros huérfanos	<code>23:00</code> , <code>06:00</code> , <code>hora de corte</code> , <code>HORA_CORTE</code> , <code>corte diario</code> : nada; <code>horaInicio</code> / <code>horaFin</code> : cuatro resultados, todos de configuración (<code>CFG_OPER_BASE</code> y los <code><input type="time"></code>) y ningún consumidor; <code>frecuenciaMin</code> : <code>60</code> : editable en Configuración > Operación y sin consumidor: el cron es <code>setInterval(..., CRON_MS)</code> (sólo con <code>modoDemo !== false</code>) y el cierre es por conteo; la ventana y la frecuencia configuradas son decorativas
Inbound	Criterios por fecha de emisión, cesión previa y lista de emisores con tags (M-10): qué tags, quién la mantiene, si es un activo nuevo del cedente	No existen en <code>CRITERIO_PRED</code> ; reclamo y NC sólo dentro de «Buena factura», no seleccionables por sí solos; los tags son del deudor (A3/A4)
Inbound	«Aceptaciones» de DTESync (M-01): qué es y si participa del filtro	No existe como campo ni regla en ningún activo ni spec (el <code>EstadoDTE</code> del A1 trae sólo <code>Reclamado</code> , <code>NotaCredito</code> , <code>FolioNotaCredito</code> ; <code>facturaDeDTE</code>); en el fuente sólo como dato sintético sin consumidor: <code>facturasDeCandidata</code> sortea <code>estado: "Aceptada" "Reclamada" "Sin acuse"</code> (única aparición), que habría que retirar o conectar al A1 si el modelo confirma que la aceptación (acuse de recibo, Ley 19.983) es criterio de candidatura
Inbound	El fuente anuncia una re-simulación automática al llegar facturas que no ocurre —en el comentario y en la bitácora, no en la pantalla del detalle—: es la re-evaluación automática que M-12 pide y la regla 14 prohíbe; corregir el comentario y los mensajes de bitácora, retirar el banner muerto, o que la decisión #1 de §15 lo contemple	La pantalla del detalle no anuncia nada: el banner «Se está recalculando la simulación con el paquete actualizado» es inalcanzable por <code>deal.actualizando && !fullPage</code> (y lo mismo su gemelo en la pestaña de deudores), porque el único <code><DealDrawer</code> pasa <code>fullPage</code> . Lo que promete la re-simulación es el comentario «el resultado re-simulado llega tras una latencia», el estado <code>actualizando: true</code> y los <code>logSys</code> «Recalculando N oportunidad(es) ... latencia estimada ...» y «Recálculo aplicado»; <code>aplicar</code> sólo anexa a <code>facturasDisponibles</code> y apaga <code>actualizando</code> , sin <code>simulado</code> , sin <code>finanzasDe</code> , sin versión
Inbound	Definición única de «Prime» para segmentar (M-05): sólo listas, o listas + nota > 4,2	Listas en <code>CRITERIO_PRED</code> y <code>capacidadDeudores</code> ; disyunción con la nota en <code>verifDecision</code> (regla 6)
Inbound	El acumulador de la corrida en <code>useState</code> se pierde al cerrar la pestaña; topes de 40 y 4 en el código	<code>spec-inbound-facturas.md</code> §10.4 y §12.4
Otorgamiento	Disposición «bloqueante del deudor» y retiro automático (M-18): contradicción 2	Llamador de <code>retirarFacturaOferta</code> con motivo distinto de <code>noConfirmada</code> : ninguno

MOTOR	PENDIENTE O DECISIÓN ABIERTA	QUÉ SE BUSCÓ Y QUÉ SE ENCONTRÓ
Otorgamiento	Excepción que deja de ser necesaria (M-21): eliminar visado, solicitud, tarea e hilo, o marcarlos «ya no aplica»	<code>huérfan</code> : sólo en Configuración > Áreas (criterios que rutean a un área borrada), nada sobre visados; <code>ya no levanta</code> , <code>ya no aplica</code> , <code>visado obsoleto</code> : nada; <code>spec-gestion-excepciones.md</code> §5.5 no lo define
Otorgamiento	Visados «versionados» (M-20): dentro de cada versión o clave estable por criterio	<code>repoVisado</code> (<code>VISADO_STATE</code>) por <code>stKey</code> ; <code>snapVersionCli</code> no lo contiene
Otorgamiento	Compuerta «sin excepciones sin justificar» en la mutación (M-19): hoy sólo en <code>ModalCurse</code> ; por el principio de la regla 24, que el cierre ya aplica al monto, falta la guarda en <code>cerrarOferta</code>	<code>cerrarOferta</code> re-comprueba sólo <code>giroCursable</code> ; no consulta <code>excPend</code> , <code>sinComentario</code> ni el visado; el botón de <code>ModalCurse</code> <code>disabled={sinComentario > 0 !gOk}</code> y <code>if (!datosCurse) return;</code> son de pantalla; la Pre-evaluación solicita sin justificar
Otorgamiento	Tres gestos de evaluación: si el modelo quiere uno solo que siempre emita versión	<code>reevaluarCliente</code> único emisor con variables frescas; <code>spec-gestion-excepciones.md</code> §4.1
Otorgamiento	El camino vivo a giro salta Pendiente Integración, VER-O1 y Operaciones N3 (desfase del sistema)	<code>useEffect</code> escribe Girada; <code>avanzarPipeline</code> sin llamador; <code>spec-ciclo-factura.md</code> §23c fila 4
Verificación	«Todas las facturas de la oferta» (M-23): las del deudor que falla o las de todos	Regla 6, que contiene la «Regla 0» de verificación (<code>spec-verificacion-facturas.md</code> §21, §4.0)
Verificación	Verificaciones «versionadas» (M-25): mismo dilema que los visados	<code>repoVerifTel</code> por factura y <code>repoVerifVeredicto</code> por deudor, fuera de la versión
Verificación	Quién y cómo registra el contacto (la operación en cero tras retirar todo ya está definida: <code>spec-ciclo-factura.md</code> §14, retirar la última factura es pérdida, citado en §11)	No está en <code>spec-verificacion-facturas.md</code> ; el vault sí (<code>DrawerVerificacion</code> , regla 53)
Verificación	Qué obtiene el contacto (M-22-bis): implementado ; pendiente sólo que <code>spec-verificacion-facturas.md</code> documente el checklist	<code>DrawerVerificacion</code> : checklist <code>{existencia, recepcion, fechaPago}</code> , rótulos, <code>completo = chk.existencia && chk.recepcion && chk.fechaPago && !!compromiso</code> , devuelve <code>checklist: chk</code> ; <code>verificarDeudor</code> y <code>VerificacionTab</code> persisten <code>{por, fecha, checklist, contacto, compromiso, respaldo, sinRespaldo, notas}</code> en <code>repoVerifTel</code> ; <code>verifFactura</code> lee <code>checks</code> ; mesa <code>CHECKS</code> ; <code>VERIF_VEREDICTO</code> sigue por deudor; la fecha de pago además como criterio previo (<code>spec-verificacion-facturas.md</code> §5 fila 6)

MOTOR	PENDIENTE O DECISIÓN ABIERTA	QUÉ SE BUSCÓ Y QUÉ SE ENCONTRÓ
Líneas	Comité que no aprueba (M-29): contradicción 4	Rechazada por el comité, <code>rechazarSolicitud</code> , <code>resolverComite</code> , <code>decisionComite</code> : nada; <code>api3EstadoProceso</code> sólo Aprobada / Observada
Líneas	La consulta A23 sin consumidor; transacción, lock y <code>requiere_resimulacion</code> inexistentes	<code>spec-ciclo-factura.md</code> §23a; <code>spec-asignacion-lineas.md</code> §6
Líneas	LF1 tras la primera operación; qué se pide al comité en estado A	<code>spec-ciclo-factura.md</code> §24
Líneas	Si el tipo de la solicitud debe seguir al motivo (M-28): hoy la solicitud automática pide siempre líneas puntuales cliente-deudor aunque el motivo sea <code>cliente</code> , <code>deudor</code> o <code>lf1</code>	<code>solicitudComiteDeOferta</code> (<code>tipo: "modificar"</code> , <code>subtipo: "agregar_deudores"</code> , <code>propGlobal: 0</code> , <code>tipoLinea: "puntual"</code>); <code>RESOLUCION_COMITE</code> sólo aporta <code>pide</code> / <code>alcance</code> ; <code>api1Inyeccion</code> no transforma
Líneas	Si la línea global del deudor (compartida por todos los clientes que le ceden) forma parte del modelo de líneas (M-27 dice cuatro) o es un control aparte	<code>asignarLineas</code> (<code>lineaDeudorDe</code>) y su condición <code>m ≤ restDeudor</code> ; <code>capacidadDeudores</code> «Nivel 3: la línea GLOBAL del deudor»; regla 7 (cinco líneas); regla 44 (A23 bloque <code>LINEA_DEUDOR</code>); regla 45 (la línea del RUT cliente ES la suma de sus líneas, ADR-0011)
Giro	El resultado de líneas no entra al criterio (M-33): que entre, o que siga sólo como LIN-01	<code>asignarGiros</code> recibe cuatro hechos, ninguno de líneas
Giro	Tercer tipo; GN disyunción o conjunción; pantalla de la asignación (§7 del spec vs regla 22)	<code>spec-modelo-giro.md</code> §2, §7
Giro	Contrato de entrega a Tesorería (payload, endpoint, idempotencia, evento)	No modelado (<code>spec-ciclo-factura.md</code> §23b); <code>recibirGiroTesoreria</code> es sólo el callback
Pricing	Tasa de referencia (M-35): el último negocio del cliente o una ventana de N negocios	«últimos negocios» (plural) o promedio: no existe; sólo <code>tasaUltimoNegocio</code> , del cliente (sintético, <code>historialComercial</code> siembra sólo por cliente), no del par; <code>spec-pricing-simulacion.md</code> §4.2 fija el último negocio, en singular, como tasa top-down, y ningún spec define una ventana ni un promedio
Pricing	Condiciones comerciales en la versión (M-36); la simulación inicial no emite versión	<code>snapVersionCli</code> y <code>huellaOperacion</code> no las contienen; <code>simularOferta</code> no toca <code>repoSimVersions</code> ; la v1 la emite retroactiva la primera re-evaluación; <code>spec-ciclo-factura.md</code> §23c fila 5
Pricing	El plazo por documento no llega al prorrateo (plazo por deudor)	Desfase medido, <code>spec-ciclo-factura.md</code> §23c fila 1

MOTOR	PENDIENTE O DECISIÓN ABIERTA	QUÉ SE BUSCÓ Y QUÉ SE ENCONTRÓ
Máquina de estados	No hay catálogo formal de transiciones: <code>spec-ciclo-factura.md</code> §20 dice que el resolver valida «contra la máquina de estados» sin definirla. La tabla de §2 es la primera enumeración, y es medida, no normativa	Transiciones repartidas entre <code>simularOferta</code> , <code>cerrarOferta</code> , <code>confirmarCierre</code> / <code>etapaTrasFirma</code> , <code>aprobarIntegracion</code> , <code>recibirGiroTesoreria</code> , <code>reabrirOperacion</code> , los dos escritores manuales <code>moverEtapa</code> y <code>moveTo</code> —con guardas parciales: <code>cesion</code> entra sin <code>integracion</code> , <code>perdida</code> sin causa, y volver a <code>oferta</code> / <code>prospeccion</code> no tiene guarda en <code>moverEtapa</code> — y los ocho escritores de pérdida
Máquina de estados	El id <code>aceptadas</code> sin escritor; si <code>Aceptada</code> y <code>Cesión</code> deben ser dos estados con la inscripción de la AEC entre medio (<code>spec-ciclo-factura.md</code> §23b)	<code>stage: "aceptadas"</code> como cadena literal: ningún escritor; la firma escribe <code>cesion</code> u <code>otorgamiento</code> (<code>patch</code> de <code>confirmarCierre</code>)
Máquina de estados	<code>clienteAcepto</code> lo escribe el canal WhatsApp, no la firma: el intent <code>cursar</code> deja la operación en <code>oferta</code> con <code>clienteAcepto: true</code> y <code>aprobacionFormalCliente</code> la da por aceptada sin portal, contra la regla 1	los dos escritores de <code>clienteAcepto: true</code> son <code>dealFirmado</code> (base en memoria) y el manejador del intent <code>cursar</code> del canal WhatsApp (<code>if (intent === "cursar" && hayOferta && !yaBoton)</code>); el <code>patch</code> de <code>confirmarCierre</code> no lo escribe; <code>aprobacionFormalCliente</code> lo lee

Documentos relacionados

DOCUMENTO	QUÉ CUBRE
Specs_Procesos/Evaluacion_Factura/spec-ciclo-factura.md	la costura entre los siete motores siguiendo a la factura; §0 mapa, §3 la reevaluación, §6 cierre, §7 post-firma, §13 pérdida, §14 deudor que no confirma, §15 reapertura, §17 versiones, §23 lo declarado y no implementado
Specs_Procesos/Evaluacion_Factura/spec-inbound-facturas.md	la corrida horaria, el filtro de calidad, las siete reglas y el dimensionamiento (§5, §6, §10.4, §12)
Specs_Procesos/Otorgamiento/spec-otorgamiento.md	el catálogo, la atribución, el ciclo del visado y el contrato del motor como servicio (§2, §3, §4, §5, §9)
Specs_Procesos/Excepciones/spec-gestion-excepciones.md	qué pasa entre que el motor levanta una excepción y un apoderado la resuelve; los cinco momentos de evaluación (§4.1), qué libera el giro (§5.3), la re-evaluación (§5.5)
Specs_Procesos/Verificacion/spec-verificacion-facturas.md	la decisión por deudor, los dos protocolos, la «Regla 0» (primera operación del cliente) y el veredicto congelado (§2.1, §4.0, §9)
Specs_Procesos/Lineas/spec-asignacion-lineas.md	las cinco líneas, la cascada, la versión y el diff, «después de aceptar sólo encoge» (§3.5, §3.7, §4.3, §6, §8.8)
Specs_Procesos/Evaluacion_Factura/spec-pricing-simulacion.md	la tasa por deudor, el catálogo de conceptos y el prorrateo (§2, §3, §4)
Specs_Procesos/Evaluacion_Factura/spec-modelo-giro.md	GE / GN, la calificación por deudor, el congelado y lo no resuelto (§2, §5, §6.1, §7)
vault/conocimiento/invariantes.md	el índice de las reglas de dominio y del contrato con el servidor citadas en la Parte III
vault/conocimiento/reglas/curse_firma_y_etapas.md	las reglas de la firma, las etapas post-firma y la integración (1, 5, 12-bis, 23, 24, 26, 28, 30, 33, 41, 43, 55, 58)
Levantamiento_Activos_Informacion.md	los activos A1–A25 (DTESync, AECSync, listas, líneas A23) y el maestro de cada campo
Integraciones/Integraciones_APIs_y_S3.md	los contratos de integración: API 1/2/3 de líneas, swagger y transporte